

Integration of Mixed-Criticality into Dynamic Task Migration for Real-Time Embedded Systems

Octavio Ivan Delgadillo Ruiz

Vollständiger Abdruck der von der Fakultät für Informatik der Technischen Universität München zur Erlangung des akademischen Grades eines

PhD (check specific degree))

genehmigten Dissertation.

Vorsitzender:

Prof. Dr. Uwe Baumgarten

Prüfende der Dissertation:

1. Prof. Dr.-Ing. Vorname Nachname
2. Prof. Dr.-Ing. Vorname Nachname

Die Dissertation wurde am 01.01.2016 bei der Technischen Universität München eingereicht und durch die Fakultät für Informatik am 10.05.2016 angenommen.

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Zusammenfassung

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Contents

Abstract	iii
Zusammenfassung	v
Contents	vii
List of Figures	xi
List of Tables	xiii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Research Methodology	4
1.5 Contributions	5
1.6 Dissertation Structure	6
2 Background and Fundamentals	7
2.1 Real-Time Systems	7
2.1.1 Scheduling Theory	8
2.1.2 Real-Time Operating Systems	9
2.2 Mixed-Criticality Systems	10
2.2.1 Criticality Levels and Standards	10
2.2.2 Scheduling and Resource Management for MCS	11
2.3 Distributed Real-Time Systems	12
2.3.1 End-to-End Timing Analysis	12
2.3.2 Heterogeneous and Edge Computing Platforms	13
2.3.3 Task Migration and Runtime Adaptation	13
2.4 Networks and Communication in Real-Time Systems	14
2.4.1 Time-Sensitive Networking	14
2.4.2 Service-Oriented Communication and Middleware	15
2.5 Summary	15

CONTENTS

3	Related Work and State of the Art	17
3.1	Scheduling and Resource Management in Real-Time Systems	17
3.1.1	Multiprocessor and Distributed Scheduling	18
3.2	Mixed-Criticality Systems	18
3.2.1	Adaptive and Multicore Mixed-Criticality Scheduling	19
3.2.2	Mixed-Criticality in Distributed and Fault-Tolerant Systems . . .	19
3.3	Distributed Real-Time Systems and Automotive Architectures	20
3.3.1	Automotive Architecture Evolution	20
3.3.2	Time-Sensitive Networking	21
3.4	Task Offloading and Orchestration in IoT and Edge Systems	21
3.5	Task Migration in Real-Time Systems	22
3.5.1	Migration Overhead and Schedulability	23
3.5.2	Fault-Tolerant Migration in Embedded Systems	23
3.5.3	Runtime Migration in Distributed Real-Time Systems	23
3.6	Machine Learning for Scheduling and Deployment	24
3.7	Prior Work at the Chair	25
3.8	Research Gap Summary	26
4	Approach	29
4.0.1	Contributions	30
4.1	General Overview	31
4.2	Overall System Architecture	34
4.3	Operating System / FreeRTOS implementation + internal behavior . . .	35
4.4	Task-Device Partitioning / Migration Planning	36
4.5	Internal Communication	36
4.6	External Communication	36
4.7	Use Cases / Test Cases	37
5	Base Operating System: Axcan OS	39
5.1	Architecture	40
5.2	Dynamic Task Deployment	43
5.2.1	Multi-Core Deployment	43
5.3	Mixed-Criticality Scheduler	44
5.4	Cross-Platform Task Loading	44
5.4.1	Hardware Abstraction Layer	46
5.4.2	User Application Requirements	46
5.5	Summary	47
6	Task Migration Implementation	49
6.1	Statefulness and Checkpoint Mechanism	49
6.2	High Criticality Strategy	50
6.3	Low Criticality Strategy	51

6.4	MEC: Virtual Devices	51
7	System Integration	53
7.1	Real-Time Constraints in the Distributed System	53
7.2	Adaption of the ML algorithm	53
7.3	Virtual and Physical Devices	53
8	Results and Evaluation	55
8.1	Test Setup	55
8.2	Discussion	55
9	Conclusion & Outlook	57
	Bibliography	59
A	Additional Stuff	65

List of Figures

2.1	Example of Schedules Generated Using Different Scheduling Policies .	8
4.1	General System Overview	30
5.1	Axcan OS Architecture	40
5.2	Example of a multicore AMP system using Axcan OS	42
5.3	OS differences between master and slave cores	42
5.4	Cross-Platform Task Compatibility	45
5.5	User Application Flow	45

List of Tables

1 Introduction

1.1 Motivation

The automotive industry is undergoing a fundamental transformation driven by the growing role of software and electronics in vehicle design and innovation. Industry analyses consistently report that the majority of automotive innovation today originates in electronic systems rather than in mechanical or materials engineering [HC16], and that spending on automotive software and electronics is projected to account for nearly half of the total vehicle bill of materials in premium vehicles by the end of this decade [KSK18]. Vehicle electrification, autonomous driving, advanced driver assistance systems (ADAS), and vehicle-to-everything (V2X) connectivity are only some of the domains where onboard computing has become central to product differentiation and safety.

A modern high-end vehicle contains upward of 100 Electronic Control Units (ECUs) connected by several kilometers of wiring harness, contributing significantly to vehicle weight and manufacturing cost [Vip18; Bau+18]. Historically, ECUs have been designed with a single-purpose philosophy: each unit is dedicated to one functional domain in order to avoid interference between tasks and to simplify safety certification [Vip+14; Vip18]. While this approach provides clear isolation and predictable behavior, it has well-known structural shortcomings. Many tasks are only active during specific and rare operating conditions, leading to chronic underutilization of processing resources across the fleet. A failure of any single ECU with no fallback mechanism can also compromise the functions it hosts. The combined effect is a system that is simultaneously over-provisioned in hardware and fragile in fault tolerance.

These inefficiencies have motivated a broader industry transition toward *ECU consolidation*: redistributing workloads from many single-purpose units to a smaller number of high-performance, multi-purpose computing platforms [Bur+18]. The emerging software-defined vehicle (SDV) concept extends this further, treating vehicle functions as software services that can be deployed, updated, and redistributed across a flexible hardware architecture at runtime [Ela+24]. Under this paradigm, the static binding between a function and a specific ECU is relaxed, and the ability to dynamically distribute and redistribute tasks at runtime becomes a key enabler of both resource efficiency and fault tolerance.

Achieving this vision, however, requires solving a problem that sits at the intersection of two fields that have historically developed largely independently: *mixed-criticality systems* and *real-time task migration*. This tension is the central intellectual challenge of

this dissertation, and it is worth stating explicitly from the outset.

Mixed-criticality (MC) systems research is fundamentally concerned with *predictability and static assurance*: given a set of tasks with heterogeneous safety requirements, expressed through standards such as ISO 26262 Automotive Safety Integrity Levels (ASILs) [Iso], how can a platform guarantee that high-criticality tasks always meet their deadlines, even under worst-case conditions, while also providing useful service to lower-criticality workloads? The entire analytical apparatus of mixed-criticality scheduling, from Vestal’s foundational model [Ves07] to more recent adaptive approaches [BD17], is built on the assumption that task parameters are known and fixed at design time, and that the platform topology does not change during execution.

Dynamic task migration, by contrast, is inherently disruptive to this assumption. Migration transfers an executing task from one node to another, necessarily interrupting its execution, serializing its state, transmitting that state over a network, and resuming execution in a different environment. Each of these steps introduces latency, consumes resources on both the source and destination node, and temporarily violates the static assumptions on which schedulability guarantees are built. In a system with hard real-time constraints, where a braking control function failing its deadline can have fatal consequences, this disruption must be managed with extreme care.

Reconciling these two requirements is non-trivial. It is not sufficient to apply existing MC scheduling on each node independently and treat migration as a separate layer: the act of migration changes the task set on each node, triggering mode transitions, modifying resource availability, and potentially invalidating schedulability guarantees that were valid before the migration event. Conversely, existing real-time migration approaches do not account for criticality-differentiated treatment of tasks, providing no mechanism to prioritize the migration of safety-critical functions over best-effort ones when resources are constrained. The intersection of these two fields, particularly in the context of ECU consolidation for automotive systems, represents a largely unexplored area in the published literature, and constitutes the primary motivation for this dissertation.

1.2 Problem Statement

Prior research at the Chair of Operating Systems of the Technical University of Munich (TUM), conducted within projects including KIA4SM, MaLSAMi, and HaCRoM, has established foundational approaches to task migration in real-time distributed systems. This includes checkpoint-based migration execution mechanisms and machine-learning-based migration planning tools capable of runtime schedulability prediction [Tai19; Utz19; Bli19]. Some of these contributions form the direct technical basis on which this dissertation builds. However, they share some critical limitations: they operate at a single, undifferentiated criticality level. All tasks are treated equivalently with respect to scheduling priority and migration policy, and no mechanism exists

to guarantee that high-criticality tasks are privileged during resource contention or migration events. Furthermore, so far no actual migration mechanism has been implemented that is compatible with execution on a real-time operating system with minimal overhead and fulfilling criticality constraints.

These limitations become untenable in the context of ECU consolidation for safety-critical automotive systems. When an ASIL-D braking function and a non-safety-relevant navigation application share the same computing platform and the same migration infrastructure, they must be treated fundamentally differently. ISO 26262 mandates that the behavior of safety-critical functions be guaranteed and verifiable under all relevant operating conditions [Iso]; a migration mechanism that might delay or interrupt such a function in the same way as a low-criticality task is not compliant with this requirement.

Four concrete open problems define the scope of this dissertation. First, the base execution platform must be extended to support dynamic loading of executables with a corresponding checkpoint-restore mechanism. Second, the execution platform must also be extended to support mixed-criticality scheduling, providing the runtime separation of concerns required for correct MC behavior before any migration mechanism is layered on top. Third, differentiated migration strategies must be designed for tasks at different criticality levels, each with appropriate timing guarantees, state preservation mechanisms, and resource overhead characteristics. The fundamental challenge here is that high-criticality migration must be fast, bounded, and formally analyzable, while low-criticality migration may trade immediacy for efficiency. Fourth, the interaction between migration events and the MC scheduler on each node must be carefully accounted for: a task arriving at a destination ECU mid-execution has different schedulability implications than a task present since system startup, and the platform design must handle this correctly.

A secondary problem concerns the role of edge computing resources in this architecture. Vehicular edge cloud (VEC) and Mobile Edge Computing (MEC) infrastructures place compute capacity in proximity to the vehicle, offering supplementary processing resources with latencies that may be acceptable for non-safety-critical workloads. Whether and how such resources can be incorporated as a third tier in a criticality-stratified migration hierarchy, and what formal conditions bound their safe use, is an exploratory research question this dissertation investigates.

1.3 Research Questions

Based on the problem statement above, this dissertation is guided by the following research questions:

- RQ1.** To what extent is task migration feasible in a distributed real-time embedded system while respecting real-time constraints, and under what conditions can timing guarantees be preserved across a migration event?

- RQ2.** How should tasks be prioritized and scheduled based on their criticality level in a system that must adapt dynamically to hardware failures and varying task load, and how can mixed-criticality scheduling be integrated with a runtime migration execution layer?
- RQ3.** What migration strategies, at both the operating system and communication levels, are most suitable for high- and low-criticality tasks, considering the trade-off between minimizing migration latency and optimizing resource utilization?
- RQ4.** Under what conditions is it feasible to offload tasks to virtual edge resources as part of a criticality-stratified migration hierarchy, and what are the implications with respect to latency, schedulability, safety assurance, and resource efficiency?

1.4 Research Methodology

This dissertation follows a design-science research methodology, proceeding through three stages: system design, prototypical implementation on real hardware, and empirical evaluation.

The first stage is *system design*. The architecture of the mixed-criticality base platform, the guest task model, and the differentiated migration strategies are developed through iterative design informed by the requirements of the target domain. Design decisions are driven by the constraints of embedded ECU-class hardware (limited memory, deterministic scheduling requirements, and the isolation demands imposed by ISO 26262) rather than by theoretical optimality criteria alone.

The second stage is *prototypical implementation*. The designed platform is realized as a working system on embedded hardware, integrating the Axcan OS mixed-criticality scheduler, an ELF-based guest task loading mechanism, checkpoint and state transfer support, and a communication infrastructure based on PTP time synchronization and SOME/IP. This prototype serves as the primary artifact through which the design is evaluated.

The third stage is *empirical evaluation*. The implemented system is evaluated through proof-of-concept demonstrations and direct measurement on the target hardware. Initial results characterize the task loading latency achievable on embedded nodes and on virtualized edge nodes respectively, establishing an empirical baseline for assessing migration feasibility across criticality tiers. Further timing and schedulability measurements are planned as the platform matures. The emphasis throughout is on grounding claims in observed system behavior on real hardware, rather than in simulation or purely analytical bounds.

1.5 Contributions

This dissertation makes the following original contributions to the state of the art in mixed-criticality real-time systems and distributed embedded computing:

1. **Mixed-criticality integration into a distributed real-time embedded platform.** The FreeRTOS-based ESFREE platform is extended to support mixed-criticality scheduling with an Asymmetric Multiprocessing (AMP) execution model on multicore ECU hardware. This provides the foundational runtime separation of criticality levels required for all subsequent contributions.
2. **Guest task architecture and ELF-based migration support.** A portable guest task architecture is defined in the form of a C library and a simplified linker script, enabling tasks to be compiled as self-describing ELF binaries that can be loaded and relocated at runtime on a target ECU. The architecture integrates checkpoint support and PTP-synchronized timing to preserve task progress and timing context across migration events.
3. **Criticality-differentiated migration strategies.** Three migration strategies are defined and implemented, each targeting a distinct criticality tier. The high-criticality (HI) strategy provides a fast, formally bounded migration path with timing guarantees sufficient for ASIL-class tasks. The low-criticality (LO) strategy employs a checkpoint-restore mechanism that prioritizes continuity of progress while minimizing interference with co-hosted high-criticality tasks. The Virtual ECU strategy defines the conditions and mechanisms under which non-safety-critical tasks may be offloaded to edge computing resources, and characterizes the associated latency and reliability constraints.
4. **Empirical characterization of task loading latency across execution tiers.** Initial measurements on the target hardware establish the task loading latency achievable by the Axcan OS ELF loading mechanism on embedded nodes, and the additional overhead introduced by containerization on virtualized edge nodes. These results provide a concrete empirical baseline for assessing the feasibility and overhead of migration across the different criticality tiers, and motivate the design choices made in the differentiated migration strategies.
5. **Mixed-criticality-aware adaptation of an ML-based migration planning toolchain.** The machine-learning-based schedulability prediction tools developed in prior theses [Tai19; Utz19; Bli19] are adapted to embedded ECU-class hardware and extended to incorporate mixed-criticality task parameters, enabling MC-aware runtime migration planning.

1.6 Dissertation Structure

The remainder of this dissertation is organized as follows.

**** TODO: Update this with correct references to chapters, keeping this structure as placeholder

Chapter 2 introduces the fundamental concepts in real-time systems, mixed-criticality scheduling, distributed embedded architectures, task migration, and communication technologies for cyber-physical systems. Chapter 3 surveys the state of the art relevant to each research question and identifies the open problems addressed by this work. Chapter 4 presents the system architecture and design, including the mixed-criticality base platform, guest task model, and migration infrastructure. Chapter 5 describes the three criticality-differentiated migration strategies in detail, including their design rationale, implementation, and initial empirical characterization. Chapter 6 presents the empirical evaluation, including schedulability analysis results, migration latency measurements, and proof-of-concept demonstrations on the target hardware platform. Chapter 7 concludes the dissertation, summarizes the contributions with respect to the research questions, and identifies directions for future work.

2 Background and Fundamentals

This chapter introduces the fundamental concepts required for understanding the research presented in this dissertation. First, the foundations of real-time systems and scheduling theory are discussed, including timing constraints, scheduling algorithms, and real-time operating systems. Afterwards, the concept of mixed-criticality systems is introduced together with the challenges associated with resource sharing and adaptability in modern embedded platforms. Subsequently, distributed real-time systems and modern automotive computing architectures are analyzed, including heterogeneous systems, edge computing, and task offloading concepts. Finally, the chapter presents the networking technologies and communication mechanisms commonly used in distributed cyber-physical systems with real-time requirements.

2.1 Real-Time Systems

Real-time systems are computing systems in which correctness depends not only on the logical correctness of the produced results, but also on the time at which these results are generated [But11]. In such systems, timing constraints are treated as first-class requirements and must be satisfied in order to guarantee correct system behavior. Real-time systems are commonly classified into hard real-time, firm real-time, and soft real-time systems depending on the consequences of missing timing deadlines. In hard real-time systems, missing a deadline may lead to catastrophic failures, making deterministic timing guarantees essential. Firm real-time systems occupy an intermediate position: a missed deadline renders the result valueless, but unlike hard real-time systems, individual misses do not necessarily trigger a system failure. Soft real-time systems, on the other hand, can tolerate occasional deadline misses with graceful degradation of quality of service.

Each task in a real-time system is typically characterized by a set of parameters: its release time (or arrival time), an execution time, a relative deadline, and—for periodic tasks—a period. The Worst-Case Execution Time (WCET) is a particularly critical parameter, as it represents an upper bound on the time a task may require under any input and execution condition. Deriving tight and safe WCET estimates is a research challenge in its own right, with techniques ranging from static analysis [Wil+08] to measurement-based approaches [Eng+03]. The inherent trade-off between safety (overestimation) and efficiency (tight bounds) is especially pronounced in heterogeneous and many-core platforms, where cache effects, pipeline speculation, and memory

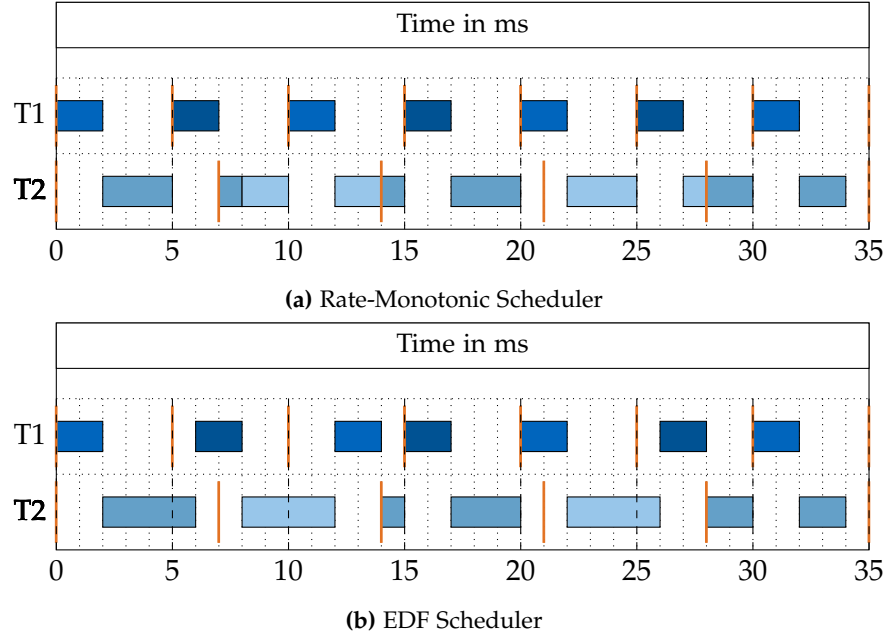


Figure 2.1: Example of Schedules Generated Using Different Scheduling Policies

contention increase analysis complexity.

Modern cyber-physical systems such as automotive platforms, industrial controllers, robotics systems, and autonomous vehicles increasingly depend on strict timing guarantees while simultaneously requiring higher computational flexibility and adaptability [Bau+18; Ela+24]. This combination introduces significant challenges for system design and resource management.

2.1.1 Scheduling Theory

Scheduling is one of the core concepts in real-time systems and refers to the process of determining the execution order of tasks while satisfying timing constraints and resource limitations. Classical real-time scheduling approaches include fixed-priority scheduling algorithms such as Rate Monotonic (RM), as well as dynamic-priority approaches such as Earliest-Deadline-First (EDF).

Liu and Layland [LL73] introduced one of the foundational scheduling models for periodic real-time tasks, establishing schedulability bounds for rate-monotonic (RM) and earliest-deadline-first (EDF) scheduling. EDF scheduling is particularly relevant due to its optimality properties for uniprocessor systems under specific assumptions: it is known to be optimal in the sense that if any algorithm can schedule a task set on a single processor, EDF can also schedule it. Fixed-priority schemes such as RM, while not theoretically optimal, offer simpler analysis and more predictable behavior in

the presence of task interactions, making them dominant in practice. A comparison between schedules generated with both RM and EDF policies is shown in Fig. 2.1, where we can notice that in many cases, the dynamic policy of EDF is able to find a schedule without missing deadlines, while RM misses deadlines for T2 in job 1.

Schedulability analysis is the formal process of determining whether a task set will always meet its deadlines under a given scheduling policy. Response Time Analysis (RTA) for fixed-priority systems [JP86] and utilization-based bounds for EDF [LL73] are well-established analytical tools. However, extending such analyses to account for blocking from shared resources, interrupt handling, and inter-task communication requires additional models such as the Sporadic Server or the Stack Resource Policy [Bak91].

As modern embedded systems evolved towards multicore and heterogeneous architectures, scheduling approaches also became more complex. Multiprocessor scheduling introduces additional challenges including task migration overhead, synchronization, resource sharing, cache interference, and communication latency [MSD10]. Broadly, multiprocessor scheduling strategies are classified into partitioned, global, and semi-partitioned (or clustered) approaches. Partitioned scheduling assigns each task permanently to one processor and applies uniprocessor analysis independently, at the cost of possible inefficiency due to bin-packing constraints. Global scheduling allows tasks to migrate freely across processors, improving utilization but introducing runtime migration overhead and making formal analysis significantly harder. Semi-partitioned and cluster-based approaches attempt to balance these trade-offs [DB11].

2.1.2 Real-Time Operating Systems

Real-Time Operating Systems (RTOSs) provide the software infrastructure required for executing time-critical tasks while maintaining deterministic system behavior. Compared to general-purpose operating systems, RTOSs prioritize predictability, bounded latency, and deterministic scheduling behavior over throughput optimization.

Common RTOS functionalities include task scheduling, inter-process communication, synchronization primitives, interrupt handling, and memory management. Depending on the system requirements, RTOSs may support static scheduling, dynamic scheduling, partitioning, or mixed-criticality execution models. Well-known examples include OSEK/VDX [Ose] and AUTOSAR Classic Platform, which are widely deployed in automotive embedded systems, as well as more general-purpose RTOSs such as FreeRTOS and VxWorks.

The AUTOSAR Adaptive Platform [Aut] represents a more recent development targeted at high-performance automotive computing nodes. Unlike the Classic Platform, which is oriented towards microcontrollers with static task configurations, the Adaptive Platform supports dynamic application deployment, service-oriented communication, and POSIX-compliant execution environments, all while maintaining defined timing and isolation guarantees. This architectural shift is particularly relevant for the integra-

tion of mixed-criticality workloads and dynamic task management in modern vehicle platforms.

In distributed and heterogeneous systems, RTOSs additionally play an important role in enabling communication, coordination, and resource sharing between different hardware nodes. The ability to support runtime adaptation and dynamic task migration increasingly represents an important requirement for modern distributed embedded systems. Hypervisor-based virtualization is increasingly used to consolidate multiple RTOS instances or mixed-criticality partitions on a single hardware platform [Hei20], providing spatial and temporal isolation with lower hardware costs.

2.2 Mixed-Criticality Systems

Mixed-criticality systems (MCS) are systems in which applications with different levels of criticality coexist on shared hardware resources [Ves07; BD17]. Such systems are especially relevant in domains including automotive systems, avionics, robotics, and industrial automation, where safety-critical and non-safety-critical applications execute simultaneously within the same platform.

Traditionally, critical systems relied on strong spatial and temporal partitioning mechanisms in order to isolate workloads with different certification requirements. While this approach improves safety and predictability, it may also lead to low resource utilization and reduced flexibility.

Vestal [Ves07] introduced one of the most influential mixed-criticality models, where tasks are associated with different Worst-Case Execution Time (WCET) estimations depending on their assurance level. Specifically, a high-criticality task carries a pessimistic (safe) WCET estimate for use under high-assurance conditions, and a more optimistic estimate for use when the system operates in a nominal low-load mode. Since then, multiple scheduling approaches have been proposed to support dynamic adaptation and graceful degradation under overload conditions. The mixed-criticality model of Baruah et al. [Bar+11a] extended Vestal's original formulation to a formal schedulability framework under EDF, providing a foundation for a significant body of subsequent research.

2.2.1 Criticality Levels and Standards

In safety-critical domains, criticality levels are often defined by domain-specific standards. In avionics, the DO-178C standard defines Design Assurance Levels (DALs) A through E for software, where DAL A represents the highest assurance requirement for functions whose failure could cause catastrophic consequences [Do1]. The corresponding hardware standard DO-254 applies analogous levels to electronic hardware design. In automotive systems, ISO 26262 defines Automotive Safety Integrity Levels (ASILs) ranging from ASIL A to ASIL D, with ASIL D representing the most stringent require-

ments for functions whose failure could lead to severe injuries or fatalities [Iso]. For general functional safety across industrial domains, IEC 61508 defines Safety Integrity Levels (SILs) 1 through 4.

These standards impose requirements not only on the software algorithms themselves but also on the processes, tools, verification methods, and evidence used during development. When components of different criticality share hardware resources, demonstrating compliance becomes significantly more complex, as the behavior of lower-criticality components may potentially interfere with higher-criticality ones. Achieving compositionality, i.e. the ability to verify system properties from verified component properties, is a central challenge in the design of mixed-criticality platforms.

2.2.2 Scheduling and Resource Management for MCS

Modern mixed-criticality systems increasingly explore adaptive resource management techniques in order to improve efficiency while preserving timing guarantees [Sal+23]. In the standard mixed-criticality scheduling model, the system can transition between a low-criticality (LO) mode and a high-criticality (HI) mode. When the observed execution time of a high-criticality task exceeds its LO-mode WCET estimate, the system switches to HI mode, abandoning or reducing the service of low-criticality tasks in order to guarantee that all high-criticality deadlines are met.

Several extensions to this basic model have been proposed. The AMC (Adaptive Mixed Criticality) scheduler [BB13] uses per-task mode switching rather than a global transition, allowing finer-grained adaptation. Elastic scheduling approaches attempt to dynamically adjust task parameters (periods, execution budgets) in response to load changes while maintaining feasibility constraints. Research on *service degradation* policies addresses the question of how best to reduce the service of lower-criticality tasks during overload, recognizing that an abrupt shutdown of lower-criticality functions may itself introduce safety risks.

Recent research has also investigated stronger isolation mechanisms and heterogeneous execution models for mixed-criticality platforms [HGB+24; TV24]. Cache coloring and cache partitioning have emerged as important mechanisms for reducing cache interference between tasks of different criticality levels executing on shared-cache multicore platforms [Man+13]. Memory bandwidth regulation and the use of memory protection units (MPUs) or memory management units (MMUs) further contribute to spatial isolation guarantees.

In distributed cyber-physical systems such as modern automotive platforms, mixed-criticality introduces additional complexity due to communication delays, distributed resource sharing, synchronization constraints, and heterogeneous hardware architectures. These systems must simultaneously maintain timing guarantees, fault tolerance, and runtime adaptability across multiple interconnected nodes. The integration of mixed-criticality scheduling with distributed resource management and task migration constitutes a central open challenge addressed in this dissertation.

2.3 Distributed Real-Time Systems

Distributed real-time systems consist of multiple computing nodes that cooperate in order to execute time-critical tasks while communicating over a network infrastructure. Such systems are commonly used in domains including automotive electronics, industrial automation, aerospace systems, and large-scale cyber-physical systems. In contrast to uniprocessor or shared-memory multiprocessor systems, distributed systems must cope with the inherent uncertainty of network communication, including variable transmission delays, packet losses, and clock synchronization errors, all while satisfying end-to-end timing constraints.

Modern automotive systems increasingly evolve from domain-oriented architectures composed of numerous Electronic Control Units (ECUs) towards centralized and zonal architectures based on high-performance computing platforms [Ela+24; Ben+23]. This transition is motivated by the growing computational requirements of software-defined functionalities, autonomous driving systems, and advanced driver assistance systems (ADAS). A zonal architecture groups ECUs by physical location in the vehicle rather than by function, with high-bandwidth Ethernet backbones connecting zone controllers to central compute nodes. This consolidation reduces wiring complexity and weight while enabling more powerful centralized processing, but also concentrates risk and demands stronger mechanisms for isolation and fault containment.

Distributed architectures provide several advantages, including scalability, fault tolerance, workload distribution, and resource sharing. However, they also introduce challenges associated with synchronization, communication latency, scheduling coordination, and distributed resource management.

2.3.1 End-to-End Timing Analysis

A fundamental challenge in distributed real-time systems is the analysis of end-to-end latency for processing chains that span multiple nodes. A *cause-effect chain* (or task chain) is a sequence of tasks executing on potentially different nodes, where each task consumes the output of the previous one. The end-to-end latency of such a chain is determined not only by the execution times of individual tasks, but also by their activation patterns, scheduling decisions on each node, and the communication delays incurred at each link.

The problem of bounding end-to-end latency has been studied through several analytical frameworks. The work of Dürr et al. [Dür+19] provides a systematic characterization of end-to-end latency semantics, distinguishing between *reaction time* (the worst-case delay from an external event to the completion of the last task in the chain) and *data age* (the maximum staleness of data observed by the final consumer). These two metrics capture different aspects of real-time correctness relevant to different application domains; for example, control systems are typically more sensitive to data age, whereas event-triggered safety responses demand short reaction times.

2.3.2 Heterogeneous and Edge Computing Platforms

Modern distributed embedded systems frequently combine heterogeneous computing resources including CPUs, GPUs, FPGAs, and specialized accelerators. Such heterogeneous platforms can improve performance and energy efficiency but also increase system complexity and scheduling challenges. The diversity of instruction set architectures, memory hierarchies, and programming models across these processing elements means that tasks cannot in general be freely migrated without adaptation of the application binary or execution environment.

The emergence of edge and Multi-access Edge Computing (MEC) architectures additionally enables the distribution of tasks between embedded devices, edge servers, and cloud infrastructure [Mec]. This paradigm supports workload offloading, adaptive deployment, and resource elasticity while reducing latency compared to centralized cloud computing approaches. In latency-sensitive cyber-physical applications, edge computing plays a particularly important role by enabling local processing of sensor data with sub-millisecond response times that would be infeasible with round-trips to a remote cloud.

Task offloading and workload distribution mechanisms are closely related to orchestration concepts commonly used in cloud and Internet-of-Things (IoT) environments. In such systems, orchestration mechanisms coordinate deployment, resource allocation, monitoring, and execution management across multiple distributed nodes. Container-based virtualization and lightweight orchestration frameworks, originally developed for cloud-native applications, are increasingly being adapted for edge and embedded contexts [Mor17], though the real-time and safety guarantees required in cyber-physical systems impose constraints that are not natively addressed by mainstream cloud orchestration tools.

2.3.3 Task Migration and Runtime Adaptation

Task migration refers to the process of transferring a task or workload from one execution node to another during system operation. Migration mechanisms may be used for fault tolerance, load balancing, resource optimization, energy management, or adaptive scheduling purposes.

Existing migration approaches range from static failover mechanisms to fully dynamic runtime migration strategies. However, implementing migration in real-time systems introduces several challenges, including state transfer, synchronization overhead, migration latency, and timing predictability [CL12; MSD10]. The migration process itself can be decomposed into several phases: (i) the *pre-migration* phase, in which the source node prepares and serializes the task state; (ii) the *transfer* phase, during which state data is transmitted to the destination node over the interconnect; and (iii) the *post-migration* phase, in which the destination node initializes execution from the transferred state. Each of these phases introduces latency that must be accounted

for in the schedulability analysis of the migrating task as well as of any tasks that share resources with it.

Checkpoint-based migration approaches periodically save a task's execution state to stable storage or shared memory, enabling resumption from the most recent checkpoint after a failure or migration event [Eln+02]. While this reduces the cost of the transfer phase by transmitting incremental state differences, it introduces regular runtime overhead for checkpointing and may not be compatible with hard real-time timing guarantees unless the checkpointing overhead is explicitly bounded and included in schedulability analysis.

In heterogeneous distributed systems, migration becomes even more complex due to architectural differences between hardware platforms, communication constraints, and execution environment compatibility. Binary incompatibility between different instruction set architectures requires either maintaining multiple compiled variants of each migratable task, recompilation at migration time, or the use of intermediate representations or virtual machines that abstract the underlying hardware. The overhead introduced by such compatibility layers must be carefully evaluated against the latency and correctness requirements of the target application.

2.4 Networks and Communication in Real-Time Systems

Communication networks represent a critical component of distributed cyber-physical systems. In real-time systems, communication mechanisms must provide deterministic behavior, bounded latency, synchronization support, and fault tolerance.

Traditional automotive systems relied heavily on fieldbus technologies such as Controller Area Network (CAN), FlexRay, and Local Interconnect Network (LIN). CAN provides a broadcast, message-priority-based medium access scheme with well-understood worst-case latency bounds, but its bandwidth is limited to 1 Mbit/s (or up to 8 Mbit/s with CAN FD). FlexRay offers higher bandwidth and a time-triggered communication model that provides stronger temporal determinism, but at higher cost and complexity. LIN is used for low-bandwidth sensor and actuator networks where cost is paramount. These technologies collectively supported the domain-oriented ECU architectures of previous vehicle generations.

2.4.1 Time-Sensitive Networking

More recent architectures increasingly incorporate Ethernet-based technologies and Time-Sensitive Networking (TSN) mechanisms in order to support higher bandwidth requirements and improved scalability [Ieeb]. TSN extends standard IEEE 802.3 Ethernet with a suite of standards developed by the IEEE 802.1 working group, collectively providing mechanisms for traffic prioritization, time synchronization, bounded latency, and deterministic communication.

Key TSN standards include IEEE 802.1AS for generalized precision time protocol (gPTP)-based clock synchronization [Ieea], IEEE 802.1Qbv for time-aware shaping (TAS), and IEEE 802.1Qbu/802.3br for frame preemption, which allows high-priority frames to interrupt the transmission of lower-priority frames. The Credit-Based Shaper (CBS) defined in IEEE 802.1Qav provides bandwidth reservation for audio-video bridging (AVB) streams. Together, these mechanisms allow TSN networks to support multiple traffic classes with heterogeneous latency and bandwidth requirements on a shared infrastructure.

TSN capabilities are especially relevant for distributed real-time systems where tasks executing on different nodes require predictable communication behavior. In the context of task migration, TSN networks can provide bounded-latency channels for state transfer, coordination messages, and monitoring traffic, enabling more predictable migration latency bounds than best-effort networks. However, the interaction between TSN traffic scheduling, task scheduling on computing nodes, and the timing constraints of distributed applications remains an active area of research [Cra+16].

2.4.2 Service-Oriented Communication and Middleware

In addition to transport-layer networking technologies, the communication middleware used in distributed real-time systems plays a significant role in system integration and runtime behavior. The AUTOSAR Adaptive Platform adopts a service-oriented communication model based on the SOME/IP protocol [Aut], which supports dynamic service discovery and method-call semantics over standard IP networks. The Data Distribution Service (DDS) middleware standard [Omg] provides a publish-subscribe communication model with Quality-of-Service (QoS) policies governing reliability, latency budgets, deadlines, and resource limits, and has seen growing adoption in autonomous and robotic systems through its use as the underlying transport in the Robot Operating System 2 (ROS2) [MKA16].

In distributed task migration and orchestration scenarios, network infrastructure additionally plays an important role in state transfer, coordination, synchronization, and runtime monitoring. Communication latency and bandwidth limitations can significantly influence migration overhead and overall system schedulability. The co-design of communication scheduling policies and task scheduling policies for distributed real-time systems with migration support constitutes one of the open research problems motivating the work presented in this dissertation.

2.5 Summary

This chapter introduced the fundamental concepts relevant to the research presented in this dissertation. Starting from the foundations of real-time systems, scheduling theory, and operating systems support, the discussion progressed to mixed-criticality models

2 Background and Fundamentals

and their associated challenges in resource sharing, mode management, and safety certification. The chapter then addressed distributed real-time system architectures, covering end-to-end timing analysis, the automotive evolution towards centralized and zonal platforms, heterogeneous and edge computing paradigms, and the specific challenges of task migration in real-time contexts. Finally, relevant communication technologies, from legacy fieldbuses to TSN and service-oriented middleware, were reviewed.

The presented concepts collectively establish the theoretical foundation required for understanding the state of the art and the research challenges discussed in the following chapter. In particular, the intersecting requirements of mixed-criticality scheduling, distributed resource management, and deterministic communication under task migration scenarios motivate the design space explored in the remainder of this dissertation.

3 Related Work and State of the Art

The increasing complexity of modern embedded and cyber-physical systems has led to extensive research on architectures that combine deterministic timing guarantees with flexibility and runtime adaptability. Classical real-time systems were designed under static assumptions regarding task sets, execution platforms, and scheduling policies. However, modern automotive and distributed embedded systems increasingly require dynamic adaptation to workload variations, heterogeneous hardware, and runtime reconfiguration. This chapter reviews the most relevant areas of research related to this dissertation, covering real-time scheduling, mixed-criticality systems, distributed embedded architectures, task migration, machine learning for scheduling, and prior work at the chair. Each section concludes by identifying the specific limitations that motivate the contributions of this dissertation.

3.1 Scheduling and Resource Management in Real-Time Systems

Foundational work in real-time scheduling established classical models for periodic task systems and provided schedulability conditions for fixed-priority and dynamic-priority scheduling algorithms [LL73; But11]. Rate Monotonic (RM) and Earliest Deadline First (EDF) scheduling remain the primary reference paradigms, with well-understood optimality properties under their respective assumptions. Response Time Analysis (RTA) [JP86] extended these results to account for blocking and shared resource access under fixed-priority scheduling, providing the analytical foundation that most industrial real-time system design still relies upon.

As systems have grown more complex, the applicability of uniprocessor scheduling theory has become increasingly limited. The transition to multicore and heterogeneous platforms introduces new sources of timing interference that classic models do not capture: shared cache contention, memory bus saturation, NUMA effects, and pipeline stalls caused by speculative execution. These phenomena invalidate simple additive compositional reasoning and have driven a substantial body of research on memory interference analysis and cache-aware scheduling for multicore platforms [Man+13; HGB+24].

3.1.1 Multiprocessor and Distributed Scheduling

Multiprocessor real-time scheduling introduces additional challenges including task migration overhead, synchronization costs, and the fundamental complexity of the multiprocessor feasibility problem. The taxonomy of partitioned, global, and semi-partitioned scheduling strategies [DB11] reflects the trade-off between analytical tractability and runtime efficiency: partitioned approaches allow independent per-core analysis but may fail to schedule task sets that global approaches can handle, while global approaches improve utilization at the cost of migration overhead and substantially harder schedulability analysis.

Megel et al. investigated migration-aware scheduling strategies aimed at minimizing preemption and migration overhead in global multiprocessor scheduling [MSD10]. Their work demonstrates that migration overhead is a first-order concern in practice, not a secondary effect, and must be explicitly modeled in schedulability analysis. However, this work addresses shared-memory multiprocessors and does not extend to distributed heterogeneous systems where migration involves network state transfer.

In distributed systems, scheduling decisions can no longer be considered independently from the communication infrastructure. End-to-end latency analysis for task chains spanning multiple nodes [Dür+19] has emerged as a key research area, particularly in the automotive domain where cause-effect chains connect sensor acquisition, computation, and actuator output across multiple ECUs. The interaction between node-local scheduling and network scheduling in TSN-based distributed systems is an active open problem [Cra+16], with joint optimization of task and network schedules shown to yield significant improvements over independent approaches.

Gap: Existing multiprocessor scheduling research addresses migration cost primarily in shared-memory settings and does not account for criticality differentiation in migration policy decisions. End-to-end timing analysis for distributed systems has advanced significantly but has not been extended to account for the transient scheduling state introduced by runtime task migration.

3.2 Mixed-Criticality Systems

The mixed-criticality model introduced by Vestal [Ves07] and subsequently formalized by Baruah et al. [Bar+11a] has generated a large body of research over the past fifteen years. The Burns and Davis survey [BD17], now in its thirteenth edition, documents the breadth of this field and identifies open problems that remain unresolved. The core mechanism of the standard MC model—mode switching from LO to HI upon a WCET overrun, with degradation or suspension of low-criticality tasks—has been extended in multiple directions.

3.2.1 Adaptive and Multicore Mixed-Criticality Scheduling

The Adaptive Mixed Criticality (AMC) scheduler proposed by Burns and Baruah [BB13] introduced per-task mode switching, allowing finer-grained response to individual task overruns rather than a global system-wide mode change. This reduces unnecessary degradation of low-criticality tasks when only a subset of high-criticality tasks approach their pessimistic WCET bounds. Subsequent work explored elastic and imprecise computation models that allow continuous service adjustment rather than binary mode transitions, improving utilization while maintaining safety guarantees.

On multicore platforms, mixed-criticality scheduling must additionally contend with shared resource interference. Cache partitioning and memory bandwidth regulation [Man+13] have been identified as essential mechanisms for providing isolation between criticality levels on shared-cache multicore hardware. The MCTI framework of Hoornaert et al. [HGB+24] provides a layered approach to task-based isolation, combining CPU regulation at the OS level with memory bandwidth control at the hypervisor level, demonstrating that strong isolation is achievable on commercial off-the-shelf (COTS) multicore platforms without specialized hardware.

Flexible and dynamic MC scheduling approaches have also been explored for scenarios where task criticality is not statically fixed [Jia+22]. Jiang et al. propose resource allocation mechanisms that dynamically adjust service levels based on runtime system state, providing a more gradual degradation model than the binary LO/HI switch. Similarly, energy-aware MC scheduling has received growing attention, with surveys documenting the trade-off between power consumption and timing guarantees in battery-constrained embedded systems [Ma+22].

3.2.2 Mixed-Criticality in Distributed and Fault-Tolerant Systems

Extending MC scheduling to distributed systems introduces the problem of mode propagation: when a high-criticality task on one node triggers a mode switch, nodes that share cause-effect chains or communication dependencies with the overrunning task must also adapt their scheduling behavior. This coordination requires either synchronous global mode transitions, which introduce communication overhead and potential blocking, or asynchronous local decisions that may lead to transient inconsistencies. Neither approach has been fully satisfactorily resolved in the literature.

Fault-tolerant mixed-criticality scheduling has been investigated by several groups. The work of Melani et al. [Mel+17] addresses fault-tolerant scheduling of MC task sets on distributed platforms using temporal redundancy, with spatial separation constraints derived from hazard analysis. Reghenzani et al. [RGF22] propose a mixed-criticality approach to fault tolerance in which criticality levels directly govern the degree of redundancy and the recovery time budget allocated to each task. These approaches demonstrate that MC and fault tolerance are complementary rather than conflicting concerns, but they assume static task allocation and do not address the case where the

execution node of a task changes at runtime.

The most directly relevant recent work to this dissertation is that of Baik et al. [BLK22], who propose a task migration and scheduling framework specifically for mixed-criticality systems in vehicle safety applications. Their approach allows low-criticality tasks to temporarily execute on neighboring nodes when their local platform is under high load, effectively extending schedulability by borrowing compute capacity, while preserving high-criticality task guarantees through deadline-based compatibility tests. However, their approach operates at the task activation level rather than providing a full execution state migration mechanism, and does not address the state transfer, checkpoint, and loading overhead involved in migrating a partially-executed task.

Gap: The intersection of MC scheduling with runtime execution state migration, where a task’s in-progress execution, including stack state and accumulated progress, is transferred to a different node, has not been addressed. Existing distributed MC work either assumes static allocation or addresses only coarse-grained failover at the task activation level.

3.3 Distributed Real-Time Systems and Automotive Architectures

Distributed real-time systems consist of multiple computational nodes connected via a communication network, and are the dominant architecture in automotive, aerospace, and industrial automation domains. The automotive domain has undergone particularly rapid architectural evolution in recent years, driven by the growing computational demands of ADAS, autonomous driving, and software-defined vehicle (SDV) functionality.

3.3.1 Automotive Architecture Evolution

The traditional domain-oriented ECU architecture, in which separate ECUs independently manage powertrain, chassis, body electronics, and infotainment, is giving way to centralized and zonal architectures [Ela+24; Ben+23]. In a zonal architecture, ECUs are grouped by physical vehicle zone rather than by function, with high-bandwidth Ethernet backbones connecting zone controllers to one or more central compute platforms. This consolidation reduces wiring harness complexity, a significant factor in vehicle weight and assembly cost, while enabling more powerful centralized processing and simpler over-the-air software updates.

The SDV concept extends this further by treating all vehicle functions as software services deployable and updatable independently of the underlying hardware. Industry adoption is accelerating: major OEMs and Tier-1 suppliers have announced centralized compute platform programs, and the AUTOSAR Adaptive Platform [Aut] provides a standardized software framework targeting these high-performance com-

puting nodes with support for dynamic service deployment and POSIX-compatible execution environments.

From a research perspective, this architectural transition surfaces a critical challenge: as more functions are consolidated onto fewer, more powerful nodes, the consequences of a node failure become more severe, making fault tolerance and runtime adaptation—including task migration—correspondingly more important.

3.3.2 Time-Sensitive Networking

Time-Sensitive Networking (TSN) has emerged as the dominant communication technology for deterministic Ethernet-based distributed embedded systems. The suite of IEEE 802.1 amendments provides mechanisms for time synchronization (802.1AS), time-aware traffic shaping (802.1Qbv), frame preemption (802.1Qbu), and credit-based shaping (802.1Qav), collectively enabling multiple traffic classes with heterogeneous latency and bandwidth requirements to coexist on shared infrastructure.

The scheduling of time-triggered traffic in TSN networks using the Time-Aware Shaper (TAS) has attracted substantial research attention. Stüber et al. [St22] provide a comprehensive survey of TAS scheduling algorithms, documenting the optimization objectives, problem formulations, and evaluation methodologies of existing approaches. A more recent systematic review and experimental study by Xue et al. [XZZ+24] evaluates 17 representative TAS scheduling methods on both simulation and real hardware testbeds, identifying that no single method dominates across all scenarios and that the interaction between network topology, traffic pattern, and scheduling algorithm is highly application-specific.

Joint scheduling of tasks and network traffic in TSN-based distributed real-time systems has been shown to offer significant improvements over independent approaches [Cra+16]. In such systems, the network schedule and the task schedule on each node are mutually dependent: the timing of task activations determines when frames are generated, while the network schedule determines when results become available for downstream tasks. This co-design problem becomes even more complex when tasks may migrate between nodes at runtime.

Gap: Existing TSN scheduling research does not address the case where the task assignment to nodes changes dynamically. Migration introduces transient traffic patterns (e.g. state transfer streams, coordination messages) that are not captured in any existing TSN scheduling model, and may require dynamic reconfiguration of the network schedule during a migration event.

3.4 Task Offloading and Orchestration in IoT and Edge Systems

In parallel to research on distributed real-time systems, a large body of work in the Internet of Things (IoT), edge computing, and fog computing has investigated the

problem of task offloading and resource orchestration across heterogeneous computing nodes. These works provide relevant conceptual models and optimization techniques, even though their primary target is best-effort or soft real-time workloads.

Mao et al. [Mao+17] studied computation offloading strategies in mobile edge computing (MEC) systems, demonstrating that optimal offloading decisions must jointly account for communication cost, computation latency, and energy consumption, and that these factors interact in non-trivial ways depending on dynamic system conditions. You et al. [You+17] extended this to joint communication and computation resource allocation, showing that network and compute constraints must be optimized simultaneously rather than independently.

Container-based virtualization has been increasingly adopted for edge computing deployments, offering process isolation and portability with lower overhead than full virtual machines. Empirical studies of checkpoint/restore in containerized environments [Tos23] demonstrate that container migration latency is dominated by the size of the process memory image and network transfer time rather than by container initialization overhead, a finding that directly informs the Virtual ECU migration strategy investigated in this dissertation. Morabito et al. [Mor17] evaluated the performance overhead of container virtualization on IoT edge devices, finding that containerization imposes measurable but manageable overhead for compute-bound workloads while adding significant latency for storage and network operations.

A fundamental limitation of IoT and edge offloading research is its assumption of relaxed or best-effort timing constraints. Offloading frameworks prioritize throughput, energy efficiency, or cost, and treat latency as a soft optimization objective rather than a hard real-time requirement. This limits the direct applicability of these approaches to safety-critical embedded systems. Nevertheless, the architectural patterns for dynamic task placement, hierarchical resource management, and state transfer developed in this body of work provide valuable reference models for the Virtual ECU migration tier explored in this dissertation, where non-safety-critical tasks may be offloaded to edge resources under controlled conditions.

Gap: Edge and IoT offloading frameworks do not provide hard real-time guarantees and do not integrate with onboard mixed-criticality scheduling. The conditions under which edge offloading is safe for tasks at the lowest criticality levels of an automotive MC system—and the latency and reliability constraints that the communication path must satisfy—have not been systematically studied.

3.5 Task Migration in Real-Time Systems

Task migration has been studied from several angles in the real-time systems literature: as a mechanism for load balancing in multiprocessor scheduling, as a fault recovery mechanism in distributed systems, and more recently as a component of adaptive orchestration frameworks.

3.5.1 Migration Overhead and Schedulability

In multiprocessor real-time scheduling, migration overhead is a well-recognized first-order concern. Megel et al. [MSD10] demonstrated that migration and preemption costs must be explicitly bounded and included in schedulability analysis, and proposed scheduling strategies that minimize the number of migrations while preserving global feasibility. Their work established that the commonly used simplification of treating migration as instantaneous is untenable in practice.

More recent work on migration-aware scheduling in heterogeneous multiprocessor systems [DB11] has confirmed that the overhead of migration—including cache repopulation, TLB invalidation, and pipeline refilling on the destination processor—can dominate the actual task execution time for short-period tasks, making migration frequency a critical design parameter.

3.5.2 Fault-Tolerant Migration in Embedded Systems

In fault-tolerant embedded systems, migration is primarily used as a recovery mechanism triggered by hardware failure. Chen and Lyu [CL12] proposed a FlexRay-based framework in which backup ECUs take over execution when the primary fails. However, their approach operates at coarse granularity: entire ECU function groups are re-assigned, rather than providing fine-grained task-level migration with state preservation. The checkpoint-restore approach to fault-tolerant migration, surveyed comprehensively by Elnozahy et al. [Eln+02], provides a principled mechanism for capturing and restoring execution state, but the original work targets message-passing cluster systems rather than resource-constrained embedded nodes with hard real-time constraints.

Checkpoint-based migration in real-time embedded contexts introduces the specific challenge that the checkpointing operation itself consumes time and memory resources, and this overhead must be budgeted in the schedulability analysis of the checkpointed task. If checkpointing is performed periodically, the checkpoint interval must be chosen to balance recovery cost (coarser intervals mean more re-execution after recovery) against runtime overhead (finer intervals impose more frequent interruptions).

3.5.3 Runtime Migration in Distributed Real-Time Systems

Research on runtime task migration specifically for distributed real-time systems remains limited. The DFT4FTT project investigated adaptive task allocation in distributed embedded systems supporting real-time, reliability, and adaptivity requirements, proposing greedy online heuristics for migrating safety-critical tasks in response to permanent faults while adjusting soft real-time task service parameters on the target node [FPY13]. This work represents one of the closest precedents to the approach developed in this dissertation, but does not address mixed-criticality scheduling or the ELF-based execution state transfer mechanisms required for heterogeneous embedded targets.

The interaction between dynamic task migration and TSN network scheduling has been highlighted as an open problem by recent work on joint task and network scheduling [Cra+16; XZZ+24]: a migration event changes the traffic pattern on the network, potentially invalidating the network schedule that was computed for the static task assignment.

Gap: No existing work provides a complete runtime task migration system for distributed embedded real-time systems that (a) differentiates migration policy and mechanism by criticality level, (b) handles the full execution state transfer including checkpoint and ELF-based loading on heterogeneous embedded targets, and (c) accounts for the schedulability implications of migration events on both source and destination nodes under a mixed-criticality scheduling framework.

3.6 Machine Learning for Scheduling and Deployment

Machine learning techniques have been increasingly applied to scheduling and resource allocation problems due to their ability to approximate complex optimization objectives that are intractable for exact methods. In the real-time scheduling domain, the primary motivation is the computational cost of exact schedulability analysis for large task sets: algorithms such as response time analysis scale poorly with the number of tasks and become infeasible for runtime use.

Early work explored neural network approaches for offline schedulability prediction and task ordering [CM94; DW04]. These approaches demonstrated that learned predictors could achieve high accuracy on synthetic task sets, but were limited to offline design-time use and did not generalize well across task set distributions.

More recent research at the chair, conducted across multiple theses [Tai19; Utz19; Bli19], applied neural network-based feasibility prediction to the migration planning problem. The key insight is that predicting schedulability, a binary decision, is faster than computing exact response times, making runtime replanning tractable even for large task sets. The trade-off is that neural network predictions are not unconditionally safe: false positive predictions (declaring a task set feasible when it is not) can occur, and the rate of such errors must be characterized and bounded for safety-critical use.

Reinforcement learning (RL) has been applied to dynamic scheduling and routing problems in TSN and edge environments, with promising results in simulation but limited deployment in safety-critical real-time systems. The fundamental challenge is that RL policies do not provide formal timing guarantees: a learned policy may perform well on average but cannot be certified to satisfy hard deadlines under all possible inputs. Bridging the gap between the performance of ML-based approaches and the certification requirements of safety-critical domains remains an open research problem.

Gap: Existing ML-based scheduling and planning tools at the chair were developed for single-criticality task sets and do not incorporate mixed-criticality parameters.

Extending one of these tools to support MC task sets, where each task has multiple WCET estimates and the feasibility prediction must account for potential mode switches, is a secondary contribution of this dissertation.

3.7 Prior Work at the Chair

Research at the Chair of Operating Systems at TUM has produced a series of projects and theses directly relevant to this dissertation. These works form the technical and conceptual foundation for task migration and adaptive execution in distributed embedded systems. However, their focus differs fundamentally from the approach presented in this dissertation: prior work primarily investigates migration as a planning, orchestration, or checkpointing problem, while assuming that a runtime execution substrate for stateful task migration is either available or abstracted away.

The KIA4SM project [EKB15] introduced a cooperative integration architecture for smart mobility systems based on Genode OS and Fiasco.OC. It demonstrated the feasibility of distributed automotive software components and introduced a two-stage migration concept (planning and execution). However, the execution stage relies on system-level abstractions and does not provide a lightweight mechanism for directly transferring and resuming task execution state on heterogeneous embedded targets.

The HaCRoM project extended checkpoint-restore mechanisms for fault-tolerant task migration in real-time systems. While this work established important empirical insights into checkpoint overhead and feasibility, it primarily focuses on state capture and recovery as a fault tolerance mechanism. The actual execution of migrated tasks is still assumed to be supported by the underlying runtime system, rather than being implemented as a minimal operating-system-level capability.

The MaLSAMi project shifted focus toward migration decision-making and planning using machine learning-based schedulability prediction. A series of theses by Taieb [Tai19], Utz [Utz19], and Blieninger [Bli19] improved the accuracy of feasibility prediction models for task allocation and migration decisions. However, these approaches operate at the level of decision support and do not address how a task is actually executed after migration, particularly in heterogeneous embedded environments.

A parallel line of work by Delgadillo [Del21] transitioned parts of the execution environment to FreeRTOS, enabling experiments closer to embedded hardware constraints. Nevertheless, the system still assumes that task execution is locally bound and does not provide a mechanism for transferring partially executed runtime state between nodes with different execution contexts.

In summary, prior work at the chair has progressively addressed migration planning, checkpointing, and feasibility prediction, but none of these systems implement migration as a first-class execution primitive within a lightweight embedded operating system. Instead, migration is treated as a higher-level orchestration decision, while the

execution of migrated tasks remains outside the scope of the runtime system itself.

None of the prior projects at the chair integrate mixed-criticality concepts into a complete runtime migration execution system. More importantly, none provide a lightweight OS-level mechanism for stateful task migration across heterogeneous embedded nodes.

The contributions of this dissertation address this gap by introducing Axcan OS, a FreeRTOS-based execution substrate that enables stateful task migration as a first-class primitive, including execution state capture, transfer, and resumption on heterogeneous embedded platforms. This runtime capability forms the foundation on which higher-level scheduling, mixed-criticality management, and planning mechanisms operate.

3.8 Research Gap Summary

The reviewed literature demonstrates substantial progress across real-time scheduling theory, mixed-criticality systems, distributed embedded architectures, communication networks, and task migration research. Classical real-time scheduling provides a strong analytical basis for fixed and dynamic priority systems under static task assumptions. Mixed-criticality research extends these models with adaptive execution modes and controlled degradation strategies. Distributed real-time systems and Time-Sensitive Networking (TSN) provide deterministic communication foundations for networked embedded systems. Finally, task migration and checkpointing techniques explore mechanisms for relocating computation, primarily in the context of fault tolerance or load balancing.

Despite these advances, existing approaches share a fundamental architectural limitation: *task execution is assumed to be bound to a fixed runtime execution context, and migration is treated as either a high-level scheduling decision or a coarse-grained fault recovery action, rather than as a continuous, OS-supported execution capability.* In all existing models, the operating system is assumed to execute tasks locally, while migration is abstracted as task reactivation, restart, or external orchestration.

This abstraction leads to a missing execution layer in current distributed embedded systems: there is no lightweight operating system substrate that enables **stateful, partially executed task migration across heterogeneous nodes as a first-class runtime primitive**. As a result, several fundamental limitations remain unresolved:

1. **Absence of OS-level execution state migration mechanisms.** Existing checkpoint-restore and migration approaches operate as external mechanisms or framework-level services. They do not integrate execution state capture, transfer, and resumption directly into a lightweight real-time operating system. Consequently, partially executed tasks cannot be transparently migrated and resumed across heterogeneous embedded nodes with minimal runtime overhead.
2. **Static execution binding in distributed real-time systems.** Current distributed

scheduling models assume a fixed mapping between tasks and execution nodes. Migration, when considered, is modeled as a discrete reassignment event rather than a runtime continuation of execution state. This prevents accurate modeling of transient execution phases, where a task is simultaneously influenced by source-side execution state, transfer latency, and destination-side scheduling constraints.

3. **Incompatibility of migration with deterministic communication schedules.** TSN-based systems assume statically known traffic patterns derived from fixed task-to-node assignments. Stateful task migration violates this assumption by introducing dynamic, time-varying communication flows, including execution state transfer and coordination traffic, which are not represented in existing offline-computed network schedules.
4. **Lack of real-time capable offloading with execution continuity.** Edge and IoT offloading frameworks support dynamic task placement across heterogeneous resources, but treat tasks as restartable computation units. They do not provide mechanisms for preserving execution continuity under hard real-time constraints, nor do they integrate with embedded operating system scheduling semantics.
5. **Separation of learning-based decision models from execution mechanisms.** Machine learning approaches for scheduling and migration focus on predicting feasibility or optimizing allocation decisions, but assume that execution after migration is handled by an external runtime system. They do not model or control the execution state transfer process, nor its impact on real-time schedulability in mixed-criticality systems.

These limitations converge to a single fundamental gap: **the absence of a lightweight operating system execution substrate that enables stateful task migration as a first-class primitive in distributed heterogeneous real-time systems, while preserving real-time constraints during execution state transfer and resumption.**

This dissertation addresses this gap through the design and implementation of Axcan OS, a FreeRTOS-based execution substrate that integrates execution state capture, transfer, and resumption directly into the runtime system. On top of this substrate, higher-level mechanisms for scheduling, mixed-criticality management, and migration planning can operate without assuming static task-to-node bindings, enabling a new class of adaptive distributed real-time systems.

4 Approach

The proposed system is a "breathable" distributed system, where breathable means that it can react at runtime to distinct situations to reconfigure itself and update a decision of where the tasks will execute, always prioritizing the execution of tasks based on their criticality. The situations that trigger this breathable behavior are mainly categorized in two types: 1) changes to the set of tasks that the system must execute (task queue) and 2) changes to the availability of computing resources. In the first category, it is assumed that all tasks have at least soft real-time behavior and they can all be categorized as periodic (need to be executed every certain time period) or sporadic (are executed upon request and at unpredictable moments) ^[**add reference**]. In the scope of this work and for simplicity, we assume two generalizations: a) all tasks can be treated as periodic tasks with a variable number of jobs (individual executions of the task that can be repeated), where specific cases with 1 total job represent sporadic tasks and an infinite number of jobs represent common periodic tasks, which could be split in batches of jobs for less critical tasks; and b) all tasks can be migrated from one device to another in between jobs. Furthermore, in the scope of this work it is assumed that tasks have no data or sequential dependencies with each other. The second category includes reacting to addition or removal of physical devices or hardware (for example, due to hardware failure or device restarts triggered by updates, etc.) and also to the availability and latency of virtual devices in the mobile-edge-cloud (for example, when driving over a highway section with MEC support).

These characteristics of the system can be summarized in the image below (add proper reference). The system behavior can be then described as follows. First, a reconfiguring situation triggers a re-distribution of the tasks that need to be executed, which are kept in the global queue. This global task queue as well as the set of both physical and virtual devices (or ECUs) are fed to a task partitioning algorithm, which decides a task-to-board mapping and then triggers the distribution of the tasks on the respective mapped device. The final step is that, depending on the previous mapping of tasks to boards, if the assigned ECU for a task changes, it is necessary to migrate its execution from the previous device (source) to the new one (target). This last step is the main objective of this research work, and in the scope of it, a few requirements should be considered to deem the implementation as successful: 1) the task has to be stopped in the source device and started in the target device, 2) the relevant task context should be also migrated, meaning that no crucial data is lost (in the case of a SLAM algorithm, the map and position would not be lost), 3) both the migration decision and execution strategies should consider the criticality of the tasks, 4) real-time behavior

4 Approach

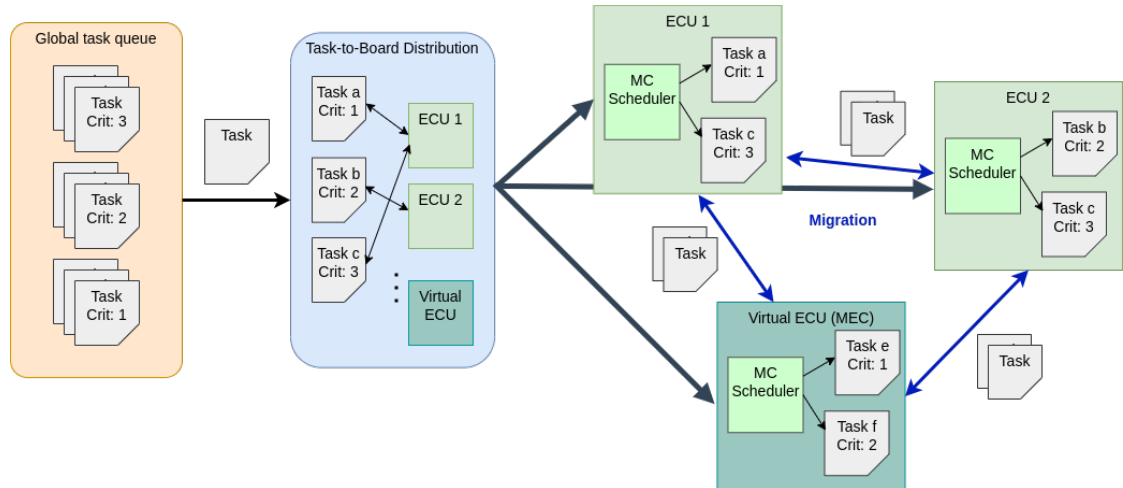


Figure 4.1: General System Overview

should be also considered in the migration strategy. It is relevant to mention that a master ECU would be responsible for both keeping the global task queue, triggering the task-to-board distribution algorithm (which itself can run on a separate ECU or a specialized ML hardware) and keeping the relevant files to start tasks when necessary.

Based on the previous list of requirements, and especially 3 and 4, at least 2 migration strategies should be implemented, since tasks with different criticalities also have different constraints to be prioritized. For example, in the case of low-criticality tasks, it might be more relevant that the task does not end in starvation, whereas for a high critical task it is crucial that the result is delivered always on-time and that, if the task needs to be paused due to a critical failure, it is brought back up in a minimal and deterministic time to keep up with the real-time constraints.

4.0.1 Contributions

This is just conceptually speaking... But in order to better place my contribution

- Task-to-Board distribution/migration planning: Toolchain for integrating ML was result of Bernhard + Robert Hamsch's thesis. My contribution: adaption to my embedded devices + include ECU, adaption to send ELF files + checkpoints, adaption to keep PTP synchronization
- ECU base system (OS): Selection of FreeRTOS as base RTOS (Bernhard, probably with input from Oliver), confirmed by me. Decision of AMP over SMP in multicore case (Bernhard + me). Some inter-processor features (probably Oliver + students). My contribution: extension of ESFREE to include MC (supported by students Mahdi and Kamran). Conceptual architecture of ELF loader, wrapper task, checkpoint keeping (implementation Yinbo).

- Guest task architecture: My contribution (supported by results of Yinbo): C task architecture in form of a library + simplified linker script (adapted from Yinbo's thesis)
- Migration strategy HI: My contribution: concept definition. Implementation partially shown in PoC by Dorota, must be extended still + maybe will require distributed shared memory implementation?
- Migration strategy LO: Concept definition (me) + implementation and PoC by Yinbo
- Migration strategy Virtual ECU (LO): Concept definition (me + Bernhard) + partial implementation and PoC analysis by Roberto + adaption and integration (me + probably Mahdi)
- Further system features: PTP + SOMEIP (Yinbo implementation + me/Bernhard concept), strategy for system functions (me), missing - network, other peripherals, interrupts, mem management?

4.1 General Overview

As mentioned before, the main objective of the proposed research is to achieve the integration of mixed criticality into the task migration process. The proper integration would require different strategies for the different criticality modes. For this reason, it is first important to define what is understood under mixed criticality in the scope of this work. While many publications consider only two criticality levels (high and low), the standards commonly describe several of them. In this work, the concept of multiple criticality levels will be used in a conceptual phase following the ASIL standard in ISO 26262, while the implementation and results will be collected for at least 2 levels in both the scheduling algorithm and the migration strategy. Whereas a model with multiple criticality levels represents real situations better over one with just two, as tasks can have intermediate criticality levels, in the scope of this work, the focus relies more on fulfilling challenges at both ends of the criticality spectrum, and assuming a mix of both strategies might apply as a solution for intermediate levels. One concept that should be covered is the consideration of different execution times according to a criticality level, as normally higher criticality levels offer pessimistic WCET estimates to ensure correct behavior occurs, but actual execution times are often lower. This is a concept that is explored in many publications and is often necessary to ensure compliance with certification authorities.

The first element needed for the system developed is the integration of a mixed-criticality scheduler for each of the physical devices. This should ensure that for any given device, the tasks with a higher criticality will never miss the deadline for a job,

4 Approach

while being more flexible for lower criticality levels. In this regard, exploring different mixed-criticality scheduling algorithms and their integration in the development platform is an important step that will allow the migration to occur and be evaluated properly. Several publications have proposed mixed-criticality scheduling algorithms, such as [Bar+11b; Fle13; ZWL17; BBD11; Li+13], and a few of them can be implemented and compared. It is also important to mention that following the state of the art in embedded devices, a multi-core implementation is desired, with asymmetrical multiprocessing (AMP) being the preferred solution, as software failures could be isolated to affect less tasks (**find a reference**). In particular, following approaches are explored (so far): hierarchical virtual deadline EDF (faulty implementation); reserved cores for 2-level MC, with LO crit cores as backup cores for HI tasks. It is assumed that this step should not affect the behavior of the virtual devices, since by concept, they can only run non highly (not necessarily only LO) critical tasks, due to the uncertainties introduced by the external communication with the MEC (explain further...).

Another important element for the distributed system to ensure its compliance to the real-time constraints is a timing protocol such as PTP, to keep the different devices synchronized and share the same notion of time when following task deadlines. This way, even when migrating a task, information on the deadlines should be very accurate. In addition to this, migration strategies must consider that transmission times can be variable and, in the ideal case, the source device should keep the original task execution until it is ensured that the target device has started the execution successfully, potentially taking longer than the deadline of a running job.

The migration planning will be adapted from previous work, namely the student theses supervised by Bernhard Blieninger. As these don't consider mixed-criticality or a multi-core implementation in the system behavior, a few changes are necessary to integrate both concepts. First, the schedulability analysis that is commonly used for selecting the best task distribution has to be extended to work with criticality levels and the use of a mixed-criticality scheduler. Here, it has to be taken into consideration whether extending and retraining the machine learning models is worth the effort, especially since this idea is still not proven as a solution and the extension with the criticality levels and other relevant information might add complexity to the net. Otherwise, implementing a mathematical schedulability analysis that considers mixed criticality is necessary. A second extension that should be performed in the planning stage is to penalize the migration of higher criticality tasks, as the migration overhead adds an additional risk for the tasks failing. Furthermore, the availability of both virtual and physical devices needs to be considered at the moment of deciding which tasks should migrate and where.

The next step in the development of this strategy is the execution of the migration, which is the core concept explored in this work. It is important to consider the necessity to meet strict timing constraints for tasks at the higher end of the criticality spectrum. ***THIS HI criticality solution is yet to be explored, Yinbo is working on the variant for

LO crit*** While the exact approach is yet to be proven as feasible, an idea for higher criticality tasks would be to let them run in a standby state in all or a subset of the total of available ECUs, and only executing it in one ECU, while storing runtime generated data either in a central unit or in a shared memory only accessible to highly critical tasks, in this way only a start/stop signal is sent to the task and the execution can be resumed quickly, also ensuring fail-safety in case the executing device fails. Also, for tasks that require redundancy (e.g. ASIL level D), it should be possible for tasks to execute actively multiple times and produce results in parallel. Another addition in this stage should be the introduction of a real-time capable communication protocol. In this sense, it should be possible to bound the migration time for this tasks, ideally in the range of a few milliseconds. These ideas could be faster to execute and allow for less variance in the time, eventually making it possible to perform formal verification, but it would be expensive if tasks at all criticality levels were to run like that, as the resources are utilized inefficiently. This should be acceptable for highly critical tasks, though, since the majority of the tasks would be assigned low to medium criticality levels.

The strategy for migrating lower criticality tasks between physical devices is prone to more flexibility, so that a wider range of tasks can be deployed with a lower impact in terms of resource efficiency, even if this would open the possibility for more erroneous behavior in the tasks. The approach currently explored for tasks with low criticality involves the transmission of precompiled task binaries and a snapshot of the execution data every time a task is distributed to an ECU and the usage of a normal TCP/IP protocol over Ethernet. ***This is Yinbo's ELF loader thesis*** With this strategy, the resource utilization in the devices is made more efficient, but the time spent for the migration is increased as the communication is slower due to the amount of information exchanged, and also less reliable due to the nature of the communication protocol.

As an optional step, the exploration of strategies for more criticality levels is yet to be done, but a few ideas are suggested. First, a combination of the two strategies mentioned could be implemented for intermediate criticality levels, such as leaving the tasks running but using a less predictable communication protocol. Another idea is to subdivide each of the criticality levels mentioned and there perform variations of the mentioned strategy. For example, in a system with a certain number of devices and 2 high criticality sublevels, the highest sublevel tasks are kept in standby on all devices, while the rest are only kept in standby in a few of the devices, making sure there is always an ECU ready for the highest criticality and using less resources for the second sublevel. In the case of lower criticality, this could be implemented in the form of giving priority to the migration and transmission of data of tasks at the higher sublevels.

Additionally, due to the consideration of virtual devices in the MEC, an additional strategy is also implemented in the scope of this work. That is, the migration of low criticality tasks from and to the MEC. For this implementation, it is assumed that the

MEC servers share some aspects of the architecture with the real hardware and that the devices can be modeled in the form of containers as their digital twins. In this way, upon availability, the master ECU can request to start a container which will run the required task. It is assumed that this behavior is allowed for low criticality tasks that can benefit from the more powerful resources available in the MEC, and that communication latency can be neglected as long as the results produced are relevant. An example of this might be a path planning algorithm for an autonomous vehicle, where the immediate safety of the vehicle is not threatened, but the algorithm can be extended with further sensor data and better hardware for performing ML tasks. *****Part of this has already been implemented by Roberto***** It is relevant in the scope of the migration, that the virtual tasks have an equivalent task implemented for the real devices, and that the task data can also be migrated between them. In this way, the task can be moved depending on the need and availability of the resources to and from the MEC.

4.2 Overall System Architecture

- Master ECU responsible for keeping track of global queue list and available hardware. Also responsible for triggering task-to-board distribution and deployment. This master ECU might have a backup ECU to meet requirements and furthermore keeps all relevant data for all tasks (precompiled task binaries / container images, context data files, criticality levels). Current implementation represents master ECU with a more powerful workstation - could be implemented to work on an embedded board similar to the other devices.
- Component performing task partitioning algorithm (might be ML-supported schedulability analysis as developed with Bernhard). Assumed to be a "black box" in most of the thesis as it is not in the scope of this research.
- Physical ECUs based on a FreeRTOS kernel extended with the functionality needed for the task migration and to comply with MC, as well as resource optimization through multicore. Some of these ECUs might be kept in a hot-standby mode or off to have backup devices while minimizing energy consumption in the vehicle
- Virtual ECUs based on a containerized minimal version of the FreeRTOS system that can execute the same tasks without the need for changes in the code
- Embedded OS Software Architecture: Extensions to the FreeRTOS Kernel - scheduler, deployment slave, inter-core communication, execution and monitoring of real-time behavior
- Task Specific Software Architecture: To be able to execute tasks with the system, developers should consider a few things when developing new tasks. For example:

definition of context data structure, loading and storing of context, external function handles, handling of system functions both in physical and virtual device...

4.3 Operating System / FreeRTOS implementation + internal behavior

Should cover following bullet points into detail when implementation is complete

- AMP multicore implementation on the Ultra96's quad-core ARM Cortex A53 (real hardware). Core 0 acts as the "master" core and cores 1-3 as "slave" cores. Core 0 interacts with master ECU and distributes tasks assigned to the board internally to the cores, following the MC (mixed-criticality) approach.
- To ensure tasks are prioritized according to criticality, MC is implemented also in the base behavior of the embedded devices. Two variants on MC implementation: 1) reserved cores with core-level EDF (avoiding conflicts between criticality levels) and 2) hierarchical virtual EDF
- Each core has "management" tasks (threads) to ensure proper functioning and dynamic functionalities: communication with master ECU (only master core), inter-core communication, task control and monitoring, and a scheduler thread
- Communication thread responsible for communicating with other cores and master ECU (only for core 0). This includes receiving instructions to execute/stop tasks, receiving and sending back task binaries (ELF) and data files, and sending back runtime data on the status of the executed tasks and the device's health.
- Task control and monitoring task involves: starting and stopping of tasks, creating new threads for the executed tasks, loading their executables, passing the stored task context to the task or storing it after task execution, collecting runtime data for the task (number of successful jobs, deadlines missed, last execution)
- Scheduler thread responsible for assigning dynamic execution priorities to tasks according to both real-time constraints and criticality (except in reserved cores approach)
- Virtual device implementation through a single FreeRTOS container running on ARM based server. Base system is a twin of the real device, but compiled for the actual processor in the server (as of now not possible to run same ELF on different ARM platform or inside the container... will explore if possible). Currently container executes task directly (without management overhead). If ELF can run on the container, it might make sense to first boot a clone with all the management capabilities and then load the required ELF inside the container

4 Approach

- System tested with simulated workload with different tasks. Two types are necessary: 1) dummy tasks allow us to test high workloads by making the CPU work without the implementation effort for different tasks, 2) actual data processing tasks allow us to test the system functionality for the migration and to demonstrate it in a tangible way
- Dummy tasks are implemented by adapting COBRA framework as it has demonstrated in the scope of my thesis to be an easy way of producing different workloads with variable real-time constraints
- Actual tasks have yet to be implemented and tested, but some ideas: image processing (e.g.: edge detection), SLAM with LIDAR... These may need to additionally deal with inputs/outputs

4.4 Task-Device Partitioning / Migration Planning

The main goal of this work is to enable the breathability of the system by making it possible to execute any task on any available virtual or physical hardware resource. However, this requires a runtime component taking decisions on where the tasks should be executed at a point in time. To achieve this, the methods explored in [**cite migration planning literature / work from Bernhard, Hamsch, etc.**] are integrated into the test setup, along with simulated task loads and hardware changes (as described in the use cases / scenarios) that will trigger the migration of tasks from one device to another.

4.5 Internal Communication

The cores communicate with each other through inter-processor interrupts and ring buffers in an allocated shared memory space, allowing for the implementation of message queues for each core. Through the implementation of "post office" library [**explain behavior and refer to Horst/Wiesboeck**], each core has a thread handling the receiving of messages in the queue, allowing the actual management task to process the message further. In general, communication between any two cores of the processor is possible using this method, but most inter-core communications in this implementation occur between the master core and a slave core.

4.6 External Communication

- Communication with other devices enabled through LWIP (lightweight IP) library, offering basic functionality: IPv4 and v6, TCP and UDP sockets, DHCP..
- Communication with master ECU through periodic TCP messages with specific format (for now using JSON)

- Transmission of files (especially ELF and context) with an implementation of SOME/IP
- Current limitation: only core 0 has control over the LWIP stack. Need to enable access for user applications from the other cores

4.7 Use Cases / Test Cases

- 1. Physical Device Failure: Tasks with HI crit have a backup device ready to continue execution, minimizing downtime. Tasks with LO crit in both backup device and faulty device migrate to available hardware. In a situation where available hardware is not able to execute all tasks in global queue, HI crit tasks are prioritized and LO crit tasks might starve or skip jobs until workload is lower or additional hardware is available again.
- 2. Increase/Decrease of Task Load: Tasks might need to be redistributed and eventually backup hardware made active. This redistribution of tasks means that migration of running tasks is likely to be triggered to achieve a stable configuration where all tasks can execute properly. In this case, migration of highly critical tasks should be avoided, except for extreme cases, as they could miss crucial deadlines or uncertainty could be introduced into their calculations and response time. In the case of a decrease in the task load, it is likely that for energy optimization, a device can be shut down, or that the task load is balanced to ensure all devices run stably.
- 3. MEC is available and a task can be supported by MEC: In this case, a task that might benefit from further resources in the MEC (like a task performing heavy ML algorithms or a task that can improve results by sensor fusion including further cameras and other information stored in the edge) is migrated to the MEC. This task can have a counterpart running on the physical hardware to ensure basic functionality is kept while enhanced functionality is integrated as long as the MEC resources are available. An example of this can be a path planning algorithm for an autonomous vehicle, where the basic functionality might be following safely a road, while the enhanced functionality might find alternative routes depending on the traffic or accident report along the road.
- 4. MEC is available and task load is critically high on ECU devices, for example, due to hardware failure (like in use case 1). In this case, as low criticality tasks would start going into starvation, they can be offloaded temporarily to the MEC, still getting time to execute, although probably introducing some latency due to external communication with the vehicle. One such case might be an infotainment service, such as streaming. In this case, it is likely that an intermediate task would have to forward the task results into the vehicle network or any resources needed.

5 Base Operating System: Axcán OS

To enable the proposed breathable distributed real-time system — capable of dynamic task migration across heterogeneous embedded nodes — a shared, minimal, and portable operating system stack is essential. This stack must satisfy several core requirements: it must be lightweight enough to operate on constrained electronic control units (ECUs) and virtualized edge platforms; it must support real-time scheduling and task isolation across mixed-criticality workloads; and it must enable dynamic execution of precompiled applications without requiring platform-specific recompilation.

Existing operating systems typically fall short on one or more of these requirements. For example, lightweight RTOSes like FreeRTOS offer deterministic scheduling and minimal overhead, but lack support for dynamic task deployment and mixed-criticality scheduling. In contrast, general-purpose real-time operating systems like PREEMPT-RT Linux provide advanced scheduling and deployment capabilities but impose a significant memory and processing footprint unsuitable for many embedded platforms.

This chapter introduces Axcán OS, a custom operating system framework built as an extension to the FreeRTOS microkernel. Axcán OS forms the runtime foundation of the breathable system by enabling cross-platform, real-time execution of position-independent applications. It provides a unified abstraction for both physical ECUs and virtual nodes hosted in mobile edge computing (MEC) environments.

Axcán OS introduces several key features that address the challenges of distributed real-time execution in the proposed system:

- **Dynamic task deployment:** a lightweight remote deployment manager receives and launches application binaries as decided by the system master node.
- **Cross-platform execution:** applications are compiled as position-independent ELF files that can be executed unmodified across compatible architectures. Furthermore, system service abstraction provide consistent interfaces for system managed functions across heterogeneous nodes.
- **Real-time capabilities:** supports various scheduling policies, including preliminary mixed-criticality strategies, and provides global time synchronization using Precision Time Protocol (PTP).

The name of the operating system comes from the Nahuatl word *axcan*, meaning “now” or “immediately,” reflecting the real-time responsiveness envisioned for the system.

5 Base Operating System: Axcan OS

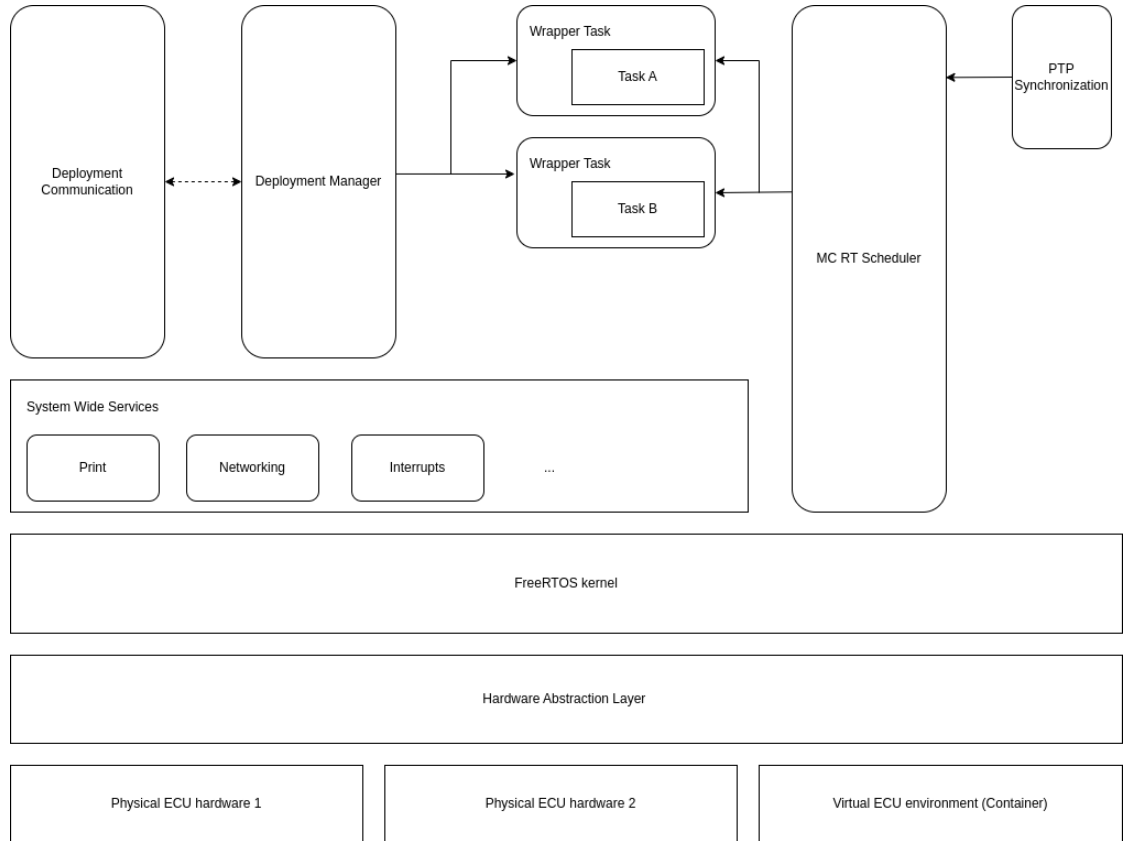


Figure 5.1: Axcan OS Architecture

The remainder of this chapter details the design, implementation, and integration of Axcan OS within the broader architecture, including its current limitations and future development directions.

5.1 Architecture

The architecture of Axcan OS is based on a microkernel approach extending the kernel of FreeRTOS as shown in Fig. 5.1. The diagram illustrates the core components of the system and their interactions. Each module contributes to fulfilling the requirements outlined in the introduction to this chapter:

- **FreeRTOS Kernel and Hardware Abstraction Layer:** These form the foundational runtime environment, providing deterministic task scheduling and hardware interfacing. Axcan OS builds on the FreeRTOS kernel to retain lightweight real-time execution.
- **Task Deployment Communication Handler:** Manages communication with the

master node, including the reception of task binaries and instructions, and the transmission of runtime status information such as health metrics or migration readiness.

- **Task Deployment Manager:** Controls the full task lifecycle, including loading ELF applications into memory, instantiating them, starting or stopping execution, and handling checkpoint data for migration purposes. The deployment manager creates instances of the wrapper task that will contain the actual applications.
- **Wrapper Task:** Acts as a container for dynamically deployed user applications. It enables runtime execution of position-independent code by handling memory allocation, real-time constraints, and state checkpointing. It also provides user tasks with access to essential system functions through controlled interfaces.
- **Extended Scheduler:** Selects tasks for executing according to a policy. Configurable for implementing different simple real-time scheduling policies, not only non-criticality-aware ones like rate-monotonic, deadline-monotonic, and earliest-deadline-first, but also mixed-criticality aware approaches.
- **System function services:** Provides a minimal API for commonly needed services such as network communication or logging. These are abstracted through the microkernel and made available to user applications via the wrapper task, compensating for the lack of shared libraries in the FreeRTOS environment.
- **PTP Synchronization:** Ensures global time alignment across nodes using the Precision Time Protocol (PTP), which is critical for maintaining timing guarantees and consistency during task migration.

Collectively, these components enable Axcan OS to serve as the runtime backbone for both physical and virtual ECU nodes within the breathable distributed system. The architecture ensures that all nodes — regardless of their underlying hardware or runtime context — can execute precompiled tasks under real-time constraints, coordinate with the master node, and participate in runtime task migration. Furthermore, Axcan OS is envisioned to work on both single-core and multi-core processors, as the multi-core approach treats each core as a sub-node, running copies of Axcan OS in every core in an asymmetric multi-processing (AMP) fashion, as shown in Fig. 5.2, on the example of the ARM Cortex-A53, the processor present by the embedded devices used in the setup of this work. In such cases, core 0 typically acts as the master core and the rest of the cores as slave cores. The architecture of the OS for master and slave cores is almost identical, except for the deployment communication, as inter-core communication and internal task partitioning is also taken into account. The master node is also responsible for the management of system services, as most of them require a dedicated thread to own the resource to avoid conflicts, therefore offering an internal wrapper interface for

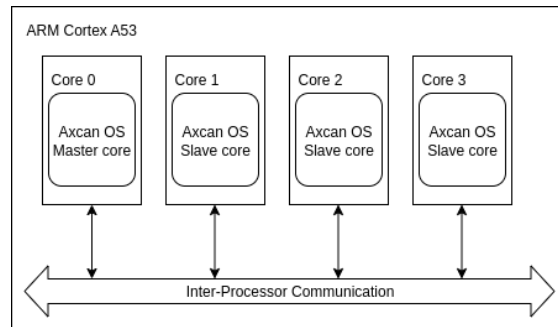


Figure 5.2: Example of a multicore AMP system using Axcan OS

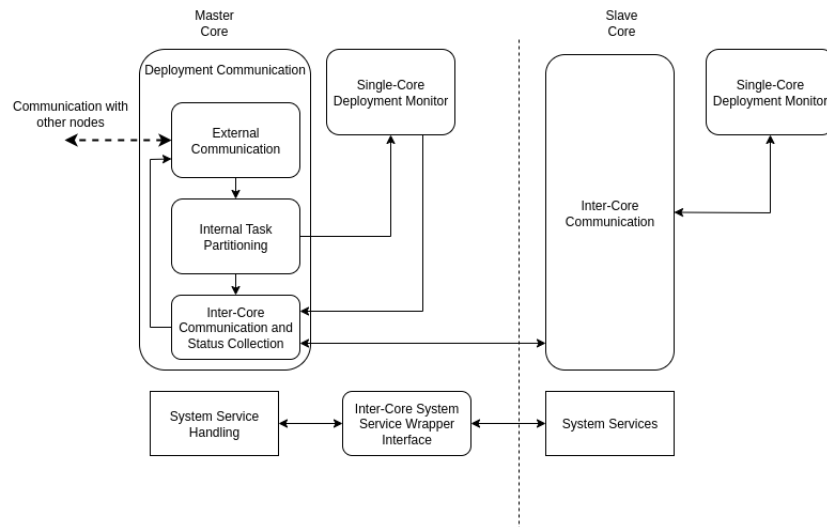


Figure 5.3: OS differences between master and slave cores

system services on the other nodes. These differences are shown in Fig. 5.3, and the internal deployment of tasks is further explained in the next section.

The next sections explain into further detail how Axcan's components interact with each other to implement the features presented in the introduction of this chapter.

5.2 Dynamic Task Deployment

The vision of the breathable system requires ECUs to be able to execute different task loads depending on dynamic needs of the system, which change depending on different factors, such as external events and the availability of hardware resources. As one ECU represents a single node of the distributed system, and the execution of tasks is coordinated by the master node, Axcan includes a pair of tasks that act as an interface for deploying tasks remotely from the master node. First, the deployment communication thread is responsible for establishing a constant communication with the master node, exchanging status information about device health and current task load and status of each task. Also, this thread is responsible for receiving instructions from the master node regarding the execution of the assigned task set, which is then kept in a shared memory block and passed on to the deployment management thread. The deployment management thread is then responsible for handling the startup of the tasks and monitoring their status periodically. The startup includes first receiving the ELF and checkpoint files necessary for task execution, then allocating and initializing the memory required by the task, and finally creating the task thread and registering it in the mixed-criticality scheduler, which is then responsible for controlling when the CPU is given to the task. The monitoring sub-thread is then responsible for periodically checking whether the task is executing correctly, how many jobs have been executed, how many deadlines have been missed, etc. Furthermore, the monitoring is responsible also for pausing / stopping the tasks when told so by the master node, and for preparing and triggering the transfer of task files to another device when migration needs to occur.

5.2.1 Multi-Core Deployment

As shown in 5.3, multi-core nodes behave in a slightly different manner. First, the master core is the only one responsible for communicating with the master node for receiving deployed tasks. After receiving a task set, the master core distributes tasks internally via a greedy partitioning algorithm, aiming for a balanced task load among all cores. This algorithm considers an additional management load for the master core. A further extension of the partitioning is envisioned to handle mixed-criticality and avoiding the clash of multiple-criticalities on independent cores. After the partitioning process has been completed, the master core sends internally the instructions to the slave cores and starts its own task load via the deployment monitor as described above.

Slave cores then replace the communication task for an inter-core communication task, responsible for performing the intermediate communication with the master core, and then perform their normal workflow with the deployment monitor task. In both cases, the monitor task works in exactly the same manner as for single-core systems. The process for sending device and task status remains almost unchanged, only having the master-core collect the status of itself and the slave cores to send a general device status message to the master node in the distributed system.

5.3 Mixed-Criticality Scheduler

The first step in achieving mixed criticality and respecting real-time behavior for the system applications is a scheduler that can achieve both goals. For this purpose, an extension of ESFREE (add reference), a project that implements a few dynamic and static real-time scheduling policies, including Earliest Deadline First (EDF), which for single core is ideally optimal (add references). This project is used as a base as the off-the-shelf FreeRTOS code only offers a priority-based scheduler, without any priority-assignment policies. In this work, the EDF policy is extended to cope with different levels of criticality by implementing the EDF with virtual deadlines for MC, as proposed in (add ref.).

**** Need to finalize implementing scheduling policies, then explain that into detail****

Furthermore, the real-time capabilities of the scheduler are enhanced by synchronizing the internal timer of the different nodes via precision-time protocol (PTP). A periodic task running at a low frequency allows then nodes to stay synchronized under a defined maximal time phase. This allows then the keeping of deadlines on any node, even when migrating tasks, by modifying the deadline to subtract that maximal phase.

5.4 Cross-Platform Task Loading

To achieve the goal of executing the same application files on both physical and virtual ECU devices, the system integrates an ELF loader. This ELF loader, implemented to a great extent in collaboration with Yinbo Zhou in the scope of his master's thesis, enables the execution of multiple tasks, packed in the form of pre-compiled ELF files and with an additional data file. This approach tackles both challenges of allowing the dynamic loading of tasks and of stateful migration to the extent desired. In order for applications to be compatible with the task loader, they need to fulfill some requirements when compiled. The implemented loader is lightweight and meant to load the same application files on multiple underlying hardware / software stacks, as shown in Fig. 5.4, as long as an Axcan OS port runs on the target and the CPU architecture is the same as the target architecture for the compiled application, which in our case is always ARM Aarch64.

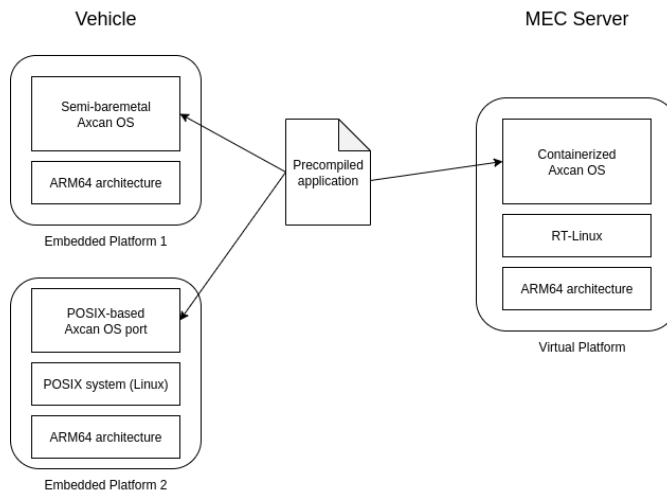


Figure 5.4: Cross-Platform Task Compatibility

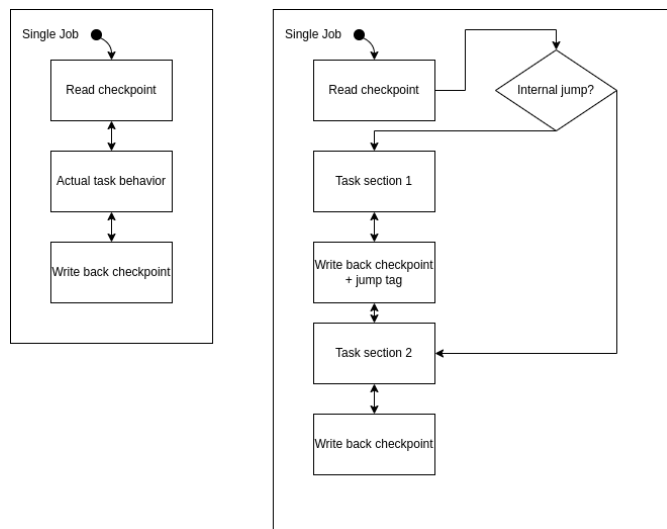


Figure 5.5: User Application Flow

5.4.1 Hardware Abstraction Layer

In order to achieve full compatibility across different platforms, an abstraction layer is added under Axcan OS. This layer ensures applications can run on any target platform independently of the hardware and software stack, apart from the architecture. The use of FreeRTOS kernel official ports already allows an abstraction layer in the form of FreeRTOS features, but due to the lightweight implementation of FreeRTOS, some features remain specific to the underlying platform, which is also undesired when aiming to load the same ELF file on different platforms. To achieve this, two core features are included as part of the extended kernel of Axcan OS. First, most of the system calls are encapsulated in wrappers, e.g.: the console output, network features, memory management, access to other peripherals, which are then offered via a standard signature that internally calls the proper underlying code. Second, the ELF loader, apart from processing the ELF and performing the needed relocations, needs to allocate memory for ELF execution, which should be aligned properly for the respective platform. This means that for hardware specific ports of FreeRTOS, which run on a bare metal system, memory can be allocated without any underlying considerations, but also requiring some memory management handling if no memory management unit (MMU) is available on the port. On the other hand, ports based on the POSIX port of FreeRTOS (e.g. the container approach running on a Linux server or even versions running as processes on top of UNIX based OSs) should take into consideration that the underlying OS is responsible for memory management, often not allowing standard *malloc* calls to hold executable code. This specific case means that alternative memory allocation functions, such as *mmap*, which are more flexible but require a more explicit memory handling in code, should be used.

5.4.2 User Application Requirements

User applications need to meet two requirements to be compatible with Axcan OS. First, they need to be compiled with the right flags, particularly by enabling position independent execution (PIE) and to avoid integration of standard C libraries and system startup functions (they should use newlib to allow for applications to be encapsulated without dynamic library linking). Second, they should use the Axcan OS header and library files, which will provide access to basic system functions and core features, such as the checkpoint mechanism, the resource table for checkpoint variable handling and the proper entrypoint handling. System services are offered through global function pointers. All those elements are initialized by the operating system and passed to user applications through a data structure in shared memory.

Furthermore, by design user applications are periodic and must define and keep track of the state-relevant variables for their further execution, meaning that the main function acts as a single job execution, and any execution context needed for a job must be loaded at the beginning of the main and stored back at the end. This represents the

minimal checkpoint version where state is only resumed from the beginning of a job. Further checkpoints can also be added in the middle of the code in the form of internal jumps that need to be executed at the start of the main function, allowing the job to resume also from internal points. These two cases are shown in Fig. 5.5.

5.5 Summary

This chapter introduced Axcan OS, the lightweight real-time operating system designed to serve as the execution environment for both physical and virtual nodes in the proposed breathable distributed system. Building upon the FreeRTOS microkernel, Axcan OS was extended with custom components to meet the core functional requirements for allowing single nodes to be integrated in the overall distributed real-time system, namely dynamic deployment of real-time tasks, task migration and execution across heterogeneous platforms and handling of tasks with different criticalities.

The modular architecture of Axcan OS, along with its internal mechanisms, as explained in this chapter, implement the backbone for the behavior of the ECU nodes, making it possible for them to work as parts of a single system. This integration of different nodes is the base for the features explained in the next chapters, such as the task migration strategies, the real-time considerations in the distributed system and the task allocation to specific nodes.

It is relevant to note that at the moment of writing, Axcan OS is implemented as a proof-of-concept framework. Further work is needed to provide a wider set of system services to user applications, and possibly to provide more efficient communication mechanisms among nodes in the distributed system.

6 Task Migration Implementation

6.1 Statefulness and Checkpoint Mechanism

In order to keep the running state of the system tasks during migration (i.e. making it stateful), a checkpoint mechanism should be implemented. In this context, checkpoint refers to a data file containing all relevant variables to store or reload a snapshot of the task's state, whereas mechanism refers to the whole process of storing and reloading the task. There is a trade-off between the precision of the checkpoint mechanism and its simplicity and amount of checkpoint data. For example, a task's state could be very precisely restored down to a specific instruction if stuff like program counter and full task context (e.g. stack and heap, and nested data structure references) can be snapshotted. Another advantage of this strategy is that tasks could be implemented without being specific to the system, but the drawback is that it might require keeping a lot of data and a more complicated implementation at kernel level. On the other hand, checkpoints can be defined in the code at some strategic steps, along with the most relevant processed variables (e.g. for an object detection algorithm after every processing step of an image). This might mean that some work could be lost after migration, but the checkpoint structure would be simple and the mechanism would involve only some code jumps, although the task would need to be specifically implemented to work with such a mechanism. The approach for the checkpoint mechanism in this work is closer to the latter, and the reasoning behind this design decision, as well as the details of its implementation are described next.

First of all, let us recall the nature of the system and the executed tasks. As mentioned before, these tasks are all assumed to be periodic, in each iteration (job) needing to process some information, possibly based on sensors or other peripherals, and generating an output, that might also depend on previous iterations. Given this behavior, it is assumed that at least, a checkpoint can be stored after a full iteration while keeping a meaningful state. Additionally, other internal checkpoints could be made by keeping jump tags if higher level of granularity is needed. During development phase, the checkpoint has to be designed in the form of a task structure which can contain all the relevant data and be stored in a file. At runtime, task reloading requires three fundamental steps, as shown in diagram...: 1) reading the existing checkpoint to get previous state, 2) performing the actual job, and 3) writing back the new state in a checkpoint. Additionally, if jump tags are stored, after retrieving the previous state, the corresponding jump is performed, while it is also important to save the checkpoint before the next sub step of the job (i.e. next jump tag), including a hint to the jump

tag. The design decision to follow this strategy was taken considering the smaller overhead at runtime and during implementation of the OS. Also, since aforementioned steps 1 and 3 (reading and writing back the state) would be the same for most simple applications, they can be pre-implemented via a library, making it only necessary to define the task state data structure, and otherwise keeping the application code fairly simple. Furthermore, flexibility for fine granularity in the checkpoint keeping is possible through the mentioned jump tags and by allowing to override the specific behavior of steps 1 and 3.

An example of the above mentioned behavior could be a SLAM (simultaneous localization and mapping) algorithm, which builds a cumulative map of the surroundings based on the received sensor data and finds the current position of the exploring agent (e.g. a vehicle) in that map. In this case, results of previous jobs (the generated map and the agent's position) are also used to generate the results of the current iteration. For this task, a minimal state could be stored by keeping track of those results, but as it is a more complex task with several steps, jump tags could be kept for example after each of the relevant steps: pre-processing sensor data, sensor fusion, identifying agent's position, filling the map with new data. This second behavior allows for a finer granularity for migrating the task while still avoiding the overhead of a full kernel-space snapshot.

While the previous paragraphs define the general behavior of the checkpoint mechanism, it is also necessary to define the way in which it is transmitted from the device currently executing the task (source) to the device that will execute next (target). As mentioned before, the user should define a data structure with all relevant data inside the task code, this will be then accessible through a void pointer from the OS in binary format, which would be further processed to share the checkpoint with the migration target. In general, defining the data structure in this way allows for a variety of implementations, as long as the data stored in it keeps its integrity, and the specific transmission approach will depend on the specific migration strategy according to the criticality of the task and the type of target device as described in the next sections.

6.2 High Criticality Strategy

*** MISSING details now, but idea is explained in approach***

- The idea is based on a premise that a shared memory approach would allow faster and less data-intensive transmission for performing the migration
- The task has 3 possible states on each device: off, running and (hot) standby. For tasks that do not require redundancy at runtime, only one (active) device (node in the distributed system) keeps them in running state, while a selection of backup nodes keep it in standby mode. To enable runtime redundancy, multiple nodes

can be active. Nodes that are neither active nor selected as backup keep task in an off state.

- Both running and hot-standby modes keep the task loaded on the system and ready to execute, but standby mode does not make task perform any work, while running mode does.
- State is kept through a shared memory approach. In the scope of this work (so far), the concept is explored using multiple cores in a single multicore processor as if they were independent nodes, with the processor memory working as the shared memory.
- Conceptually, this memory can represent a distributed shared memory (DSM), which could be further integrated using a real-time communication protocol to keep time-bound behavior. Example of this approach might be... (Do proper referencing...) <https://doi-org.eaccess.tum.edu/10.1007/s10586-017-1140-9> <https://dl.acm.org/doi/abs/10.1007/107-1140-9>
- The idea would be to keep a fixed reference to this space in memory for all running and standby instances, which would be read before and updated after each executed job.

6.3 Low Criticality Strategy

- Based greatly on Yinbo's thesis
- Here, idea is that tasks can be precompiled to ELF files which can be loaded dynamically upon request
- Task data are stored in a separate file that can be loaded with the ELF and later stored upon request
- Explain how tasks must be compiled and prepared to provide valid ELF files.
- Loader would work with any scheduling algorithm, ensuring that MC can be respected in the system

6.4 MEC: Virtual Devices

- Based greatly on Roberto's thesis
- Here, idea is that the MEC can host containerized versions of the devices – Implementation specific details for the containers are out of scope in the thesis, as they are part of Bernhard's idea

6 Task Migration Implementation

- Explain briefly that it is assumed the implementation of the containerized version of the tasks is fully implemented in the scope of future work
- Explain how the migration to and from the cloud works
- Explain how the migration to and from the MEC is integrated into the system architecture, especially how it is considered when taking a decision to migrate to MEC or to another physical device
- Discuss missing implementation details

The work presented in (cite ROberto's thesis) showed the possibility of containerizing a POSIX port of FreeRTOS and running it on a Linux server. Following this concept, Axcan OS was ported to make it compatible with such a container approach, making it possible to load on the virtual devices exactly the same ELF executables that are loaded for the real embedded devices, avoiding the need for recompiling the tasks. Furthermore, the features of Axcan OS allow the tasks to access the system services in a seamless manner.

The container-compatible POSIX port of Axcan OS requires a different management of memory and peripherals, especially due to the restrictions introduced by the Linux kernel, for example due to the exception level or the rights assigned to memory allocated with functions like malloc or mmap. However, Axcan OS offers an abstraction layer that offers the features to the user applications on the same way on both platforms.

7 System Integration

7.1 Real-Time Constraints in the Distributed System

7.2 Adaption of the ML algorithm

7.3 Virtual and Physical Devices

8 Results and Evaluation

8.1 Test Setup

8.2 Discussion

9 Conclusion & Outlook

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Bibliography

- [Aut] *AUTOSAR Adaptive Platform: Main Requirements, Release 22-11*. Tech. rep. Available at <https://www.autosar.org>. AUTOSAR Consortium, 2023.
- [Bak91] T. P. Baker. "Stack-Based Scheduling of Real-Time Processes." In: *Real-Time Systems* 3.1 (1991), pp. 67–99. doi: 10.1007/BF00365393.
- [Bar+11a] S. Baruah, V. Bonifaci, G. D'Angelo, A. Marchetti-Spaccamela, S. van der Ster, and L. Stougie. "Preemptive Uniprocessor Scheduling of Mixed-Criticality Sporadic Task Systems." In: *Proceedings of the 23rd Euromicro Conference on Real-Time Systems (ECRTS)*. 2011, pp. 145–154. doi: 10.1109/ECRTS.2011.20.
- [Bar+11b] S. K. Baruah, V. Bonifaci, G. D'Angelo, A. Marchetti-Spaccamela, S. van der Ster, and L. Stougie. "Mixed-Criticality Scheduling of Sporadic Task Systems." In: *Algorithms – ESA 2011*. Ed. by C. Demetrescu and M. M. Halldórsson. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 555–566. ISBN: 978-3-642-23719-5.
- [Bau+18] M. Baunach, R. Martins Gomes, M. Malenko, F. Mauroner, L. Batista Ribeiro, and T. Scheipel. "Smart mobility of the future - a challenge for embedded automotive systems." In: *e & i Elektrotechnik und Informationstechnik*. Vol. 135. 4. 2018, pp. 304–308. doi: 10.1007/s00502-018-0623-6.
- [Bau+18] M. Baunach, R. M. Gomes, M. Malenko, F. Mauroner, L. B. Ribeiro, and T. Scheipel. "Smart Mobility of the Future – A Challenge for Embedded Automotive Systems." In: *e & i Elektrotechnik und Informationstechnik* 135.4 (2018), pp. 304–308.
- [BB13] A. Burns and S. Baruah. "Sustainability in Real-Time Scheduling." In: *Journal of Computing Science and Engineering* 7.4 (2013), pp. 235–247. doi: 10.5626/JCSE.2013.7.4.235.
- [BBD11] S. Baruah, A. Burns, and R. Davis. "Response-Time Analysis for Mixed Criticality Systems." In: *2011 IEEE 32nd Real-Time Systems Symposium*. 2011, pp. 34–43. doi: 10.1109/RTSS.2011.12.
- [BD17] A. Burns and R. I. Davis. "A Survey of Research into Mixed Criticality Systems." In: *ACM Computing Surveys* 50.6 (2017).
- [Ben+23] S. Bensalem et al. "High-Performance Computing Platforms for Autonomous Vehicles: Architectures and Challenges." In: *IEEE Access* 11 (2023), pp. 108233–108260.

Bibliography

- [Bli19] B. Blieninger. *Online Machine Learning for improved decision making regarding the migration of automotive software components at runtime*. 2019.
- [BLK22] J. Baik, J. Lee, and K. Kang. "Task Migration and Scheduler for Mixed-Criticality Systems." In: *Sensors* 22.5 (2022), p. 1926. DOI: 10.3390/s22051926.
- [Bur+18] O. Burkacky, J. Deichmann, G. Doll, and C. Knochenauer. *Rethinking Car Software and Electronics Architecture*. Tech. rep. McKinsey & Company, Feb. 2018.
- [But11] G. C. Buttazzo. *Hard Real-Time Computing Systems*. 3rd ed. Springer, 2011.
- [CL12] Y.-Y. Chen and C.-M. Lyu. "ECU-Level Fault-Tolerant Framework for Safety-Critical FlexRay Network Systems." In: *International Conference on ICT Convergence*. 2012, pp. 553–558.
- [CM94] C. Cardeira and Z. Mammeri. "Neural Network-Based Approach for Real-Time Scheduling." In: *Proceedings of the IEEE Workshop on Neural Networks for Signal Processing*. 1994, pp. 576–585. DOI: 10.1109/NNSP.1994.366059.
- [Cra+16] S. S. Craciunas, R. S. Oliver, M. Chmelik, and W. Steiner. "Scheduling Real-Time Communication in IEEE 802.1Qbv Time Sensitive Networks." In: *Proceedings of the 24th International Conference on Real-Time Networks and Systems (RTNS)*. 2016, pp. 183–192. DOI: 10.1145/2997465.2997470.
- [DB11] R. I. Davis and A. Burns. "A Survey of Hard Real-Time Scheduling for Multiprocessor Systems." In: *ACM Computing Surveys* 43.4 (2011), 35:1–35:44. DOI: 10.1145/1978802.1978814.
- [Del21] O. I. Delgadillo Ruiz. *Hardware-in-the-Loop Test Setup for a Generalistic Approach to Machine-Learning-Based Schedulability Analysis*. 2021.
- [Do1] DO-178C: *Software Considerations in Airborne Systems and Equipment Certification*. Tech. rep. RTCA DO-178C / EUROCAE ED-12C. RTCA / EUROCAE, 2011.
- [Dür+19] M. Dürr, D. Ziegenbein, A. Hamann, and R. Ernst. "End-To-End Timing Analysis of Sporadic Cause-Effect Chains in Distributed Systems." In: *ACM Transactions on Embedded Computing Systems* 18.5s (2019), 58:1–58:24. DOI: 10.1145/3358181.
- [DW04] J. Dominguez and A. Wellings. "Applying Neural Networks to Fixed-Priority Scheduling of Sporadic Task Sets." In: *Proceedings of the 10th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 2004. DOI: 10.1109/RTAS.2004.1247674.
- [EKB15] S. Eckl, D. Krefft, and U. Baumgarten. "KIA4SM - Cooperative Integration Architecture for Future Smart Mobility Solutions." In: *Conference on Future Automotive Technology (CoFAT)*. Vol. 4. Apr. 2015.

- [Ela+24] M. Elamin et al. "Software-Defined Vehicle Architecture: A Survey and Research Directions." In: *IEEE Access* 12 (2024), pp. 42115–42148.
- [Eln+02] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson. "A Survey of Rollback-Recovery Protocols in Message-Passing Systems." In: *ACM Computing Surveys* 34.3 (2002), pp. 375–408. DOI: 10.1145/568522.568525.
- [Eng+03] J. Engblom, A. Ermedahl, M. Sjödin, J. Gustafsson, and H. Hansson. "Worst-Case Execution-Time Analysis for Embedded Real-Time Systems." In: vol. 4. 4. 2003, pp. 437–455. DOI: 10.1007/s10009-002-0080-7.
- [Fle13] T. D. Fleming. *Extending Mixed Criticality Scheduling*. Tech. rep. The University of York, 2013.
- [FPY13] L. L. Ferreira, L. M. Pinho, and P. M. Yomsi. "Task Migration for Fault-Tolerance in Mixed-Criticality Embedded Systems." In: *Work-in-Progress Proceedings of the Euromicro Conference on Real-Time Systems (ECRTS)*. DFT4FTT project. 2013.
- [HC16] C. Hainz and A. Chauhan. *Automotive Change Drivers for the Next Decade*. Tech. rep. EY Global Automotive & Transportation Sector, 2016.
- [Hei20] G. Heiser. "Hypervisors for Consumer Electronics." In: *Proceedings of the IEEE Consumer Electronics Show Conference*. 2020. DOI: 10.1109/ICCE46568.2020.9042985.
- [HGB+24] D. Hoornaert, G. Ghaemi, A. Bastoni, et al. "MCTI: Mixed-Criticality Task-Based Isolation." In: *Real-Time Systems* 60 (2024), pp. 328–365.
- [Ieea] *IEEE Std 802.1AS: Timing and Synchronization for Time-Sensitive Applications*. Tech. rep. IEEE 802.1AS-2020. IEEE, 2020.
- [Ieeb] *IEEE Std 802.1Q: Bridges and Bridged Networks*. Tech. rep. IEEE 802.1Q-2022. IEEE, 2022.
- [Iso] *ISO 26262: Road Vehicles—Functional Safety, Parts 1–12*. Tech. rep. Second Edition. International Organization for Standardization, 2018.
- [Jia+22] X. Jiang, T. Sha, D. Liu, J. Chen, C. Chen, and K. Huang. "Flexible and Dynamic Scheduling of Mixed-Criticality Systems." In: *Sensors* 22.19 (2022), p. 7528. DOI: 10.3390/s22197528.
- [JP86] M. Joseph and P. Pandya. "Finding Response Times in a Real-Time System." In: *The Computer Journal* 29.5 (1986), pp. 390–395. DOI: 10.1093/comjnl/29.5.390.
- [KSK18] F. Kuhnert, C. Stürmer, and A. Koster. *Five Trends transforming the Automotive Industry*. Tech. rep. PricewaterhouseCoopers GmbH Wirtschaftsprüfungsgesellschaft, 2018.

Bibliography

- [Li+13] L. Li, R. Li, L. Huang, R. Wu, and L. Zeng. “A New RTA Based Scheduling Algorithm for Mixed-Criticality Systems.” In: *2013 IEEE 16th International Conference on Computational Science and Engineering*. 2013, pp. 722–729. doi: 10.1109/CSE.2013.111.
- [LL73] C. L. Liu and J. W. Layland. “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment.” In: *Journal of the ACM* 20.1 (1973), pp. 46–61.
- [Ma+22] Y. Ma, W. Liu, T. Gong, Y. Jiang, M. Xu, and K. G. Shin. “A Survey of Energy-Aware Scheduling in Mixed-Criticality Systems.” In: *Journal of Systems Architecture* 126 (2022), p. 102476. doi: 10.1016/j.sysarc.2022.102476.
- [Man+13] R. Mancuso, R. Pellizzoni, M. Caccamo, L. Sha, and H. Yun. “WCET (m) Estimation in Multi-Core Systems Using Single Core Equivalence.” In: *Proceedings of the 25th Euromicro Conference on Real-Time Systems (ECRTS)*. 2013, pp. 174–183. doi: 10.1109/ECRTS.2013.27.
- [Mao+17] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief. “A Survey on Mobile Edge Computing: The Communication Perspective.” In: *IEEE Communications Surveys & Tutorials* 19.4 (2017), pp. 2322–2358. doi: 10.1109/COMST.2017.2745201.
- [Mec] *Mobile Edge Computing (MEC): Framework and Reference Architecture*. Tech. rep. GS MEC 003. V3.1.1. ETSI, 2022.
- [Mel+17] A. Melani, M. Bertogna, V. Bonifaci, A. Marchetti-Spaccamela, and G. C. Buttazzo. “Response-Time Analysis of Conditional DAG Tasks in Multiprocessor Systems.” In: *Proceedings of the 29th Euromicro Conference on Real-Time Systems (ECRTS)*. 2017, pp. 1–22. doi: 10.4230/LIPIcs.ECRTS.2017.1.
- [MKA16] Y. Maruyama, S. Kato, and T. Azumi. “Exploring the Performance of ROS2.” In: *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*. 2016. doi: 10.1145/2968478.2968502.
- [Mor17] R. Morabito. “Virtualization on Internet of Things Edge Devices with Container Technologies: A Performance Evaluation.” In: *IEEE Access* 5 (2017), pp. 8835–8850. doi: 10.1109/ACCESS.2017.2704444.
- [MSD10] T. Megel, R. Sirdey, and V. David. “Minimizing Task Preemptions and Migrations in Multiprocessor Optimal Real-Time Schedules.” In: *31st IEEE Real-Time Systems Symposium*. 2010, pp. 37–46.
- [Omg] *Data Distribution Service (DDS) Specification, Version 1.4*. Tech. rep. OMG Document formal/2015-04-10. Object Management Group (OMG), 2015.
- [Ose] *OSEK/VDX Operating System Specification 2.2.3*. Tech. rep. Available at <https://www.osek-vdx.org>. OSEK Group, 2005.

- [RGF22] F. Reghenzani, Z. Guo, and W. Fornaciari. "A Mixed-Criticality Approach to Fault Tolerance: Integrating Responses to Faults with Mode Changes." In: *IEEE Transactions on Computers* 71.11 (2022), pp. 2917–2930. doi: 10.1109/TC.2022.3144113.
- [Sal+23] M. Salehi et al. "Adaptive Mixed-Criticality Scheduling for Energy-Aware Real-Time Systems." In: *Journal of Systems Architecture* 138 (2023), p. 102867.
- [St22] T. Stüber, L. Osswald, S. Lindner, and M. Menth. "A Survey of Scheduling Algorithms for the Time-Aware Shaper in Time-Sensitive Networking (TSN)." In: *arXiv preprint arXiv:2211.10954* (2022). Published version: *IEEE Communications Surveys & Tutorials*.
- [Tai19] S. Taieb. *Deep Learning Based Schedulability Analysis for Migration of Software Components at Runtime*. 2019.
- [Tos23] A. Tomic. "Run-Time Application Migration Using Checkpoint/Restore in Userspace." In: *arXiv preprint arXiv:2307.12113* (2023). Empirical study of CRIU-based container migration for edge computing scenarios.
- [TV24] E. Tinto and T. Vardanega. "Providing Spatial Isolation for Mixed-Criticality Systems." In: *Journal of Systems Architecture* 154 (2024), p. 103234.
- [Utz19] T. Utz. *Vergleich von heuristischen und Deep Learning basierten Schedulability Analyse Verfahren für die Migration von Softwarekomponenten zur Laufzeit*. 2019.
- [Ves07] S. Vestal. "Preemptive Scheduling of Multi-Criticality Systems with Varying Degrees of Execution Time Assurance." In: *Proceedings of the 28th IEEE International Real-Time Systems Symposium*. 2007, pp. 239–243.
- [Vip+14] K. Vipin, S. Shreejith, S. A. Fahmy, and A. Easwaran. "Mapping Time-Critical Safety-Critical Cyber Physical Systems to Hybrid FPGAs." In: *2014 IEEE International Conference on Cyber-Physical Systems, Networks, and Applications*. 2014, pp. 31–36. doi: 10.1109/CPSNA.2014.14.
- [Vip18] K. Vipin. "CANNoC: An open-source NoC architecture for ECU consolidation." In: *2018 IEEE 61st International Midwest Symposium on Circuits and Systems (MWSCAS)*. 2018, pp. 940–943. doi: 10.1109/MWSCAS.2018.8624006.
- [Wil+08] R. Wilhelm et al. "The Worst-Case Execution-Time Problem—Overview of Methods and Survey of Tools." In: *ACM Transactions on Embedded Computing Systems* 7.3 (2008), 36:1–36:53. doi: 10.1145/1347375.1347389.
- [XZZ+24] C. Xue, T. Zhang, Y. Zhou, et al. "Real-Time Scheduling for IEEE 802.1Qbv Time-Sensitive Networking (TSN): A Systematic Review and Experimental Study." In: *ACM Computing Surveys* (2024). Published online ahead of print. doi: 10.1145/3736715.

Bibliography

- [You+17] C. You, K. Huang, H. Chae, and B.-H. Kim. “Energy-Efficient Resource Allocation for Mobile-Edge Computation Offloading.” In: *IEEE Transactions on Wireless Communications* 16.3 (2017), pp. 1397–1411. doi: 10.1109/TWC.2016.2633522.
- [ZWL17] X. Zhao, Y. Wei, and W. Li. “The Improved Earliest Deadline First with Virtual Deadlines Mixed-Criticality Scheduling Algorithm.” In: *2017 IEEE International Symposium on Parallel and Distributed Processing with Applications and 2017 IEEE International Conference on Ubiquitous Computing and Communications (ISPA/IUCC)*. 2017, pp. 444–448. doi: 10.1109/ISPA/IUCC.2017.00072.

A Additional Stuff

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.